

# 高级操作系统自学笔记

本笔记依据 2022 之后的考试题库内容，不涉及更早的题库，但包含师 PPT 里大部分习题，不涉及深入理解得内容，只针对能够快速理解并通过考试。计算题详述会过于啰嗦，以便本人复习时能看懂，如有错误，还请指正，谢谢！

——江一鸣

## 一，计算机系统概述

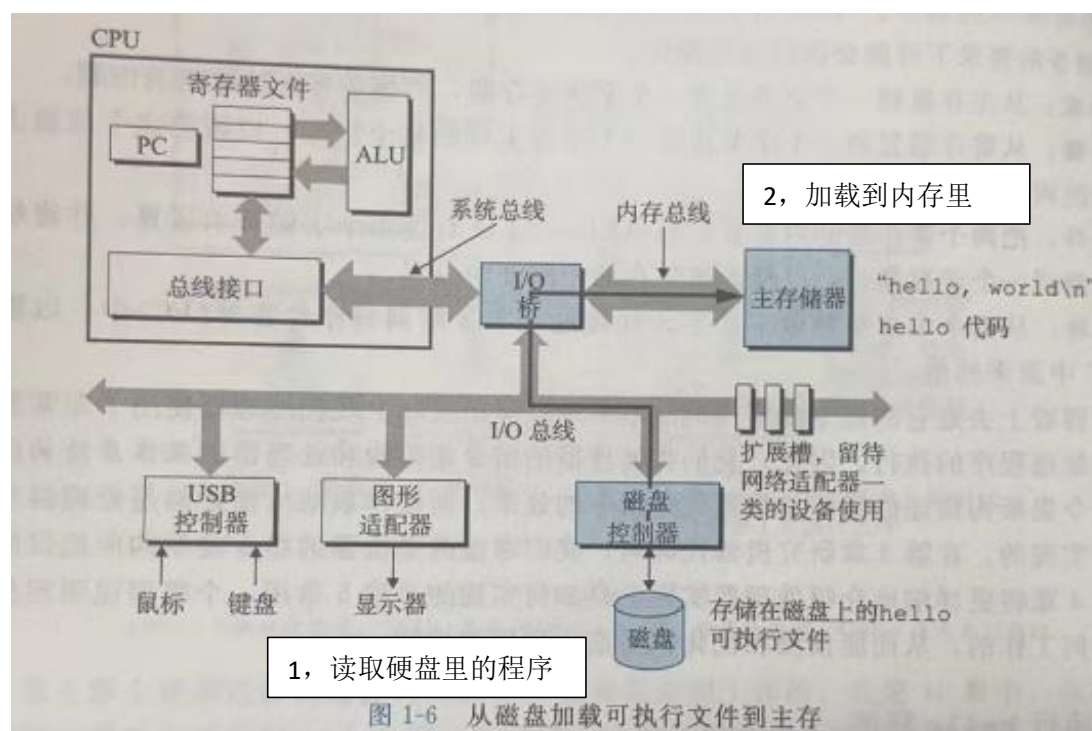
下图为计算机硬件为物理配件



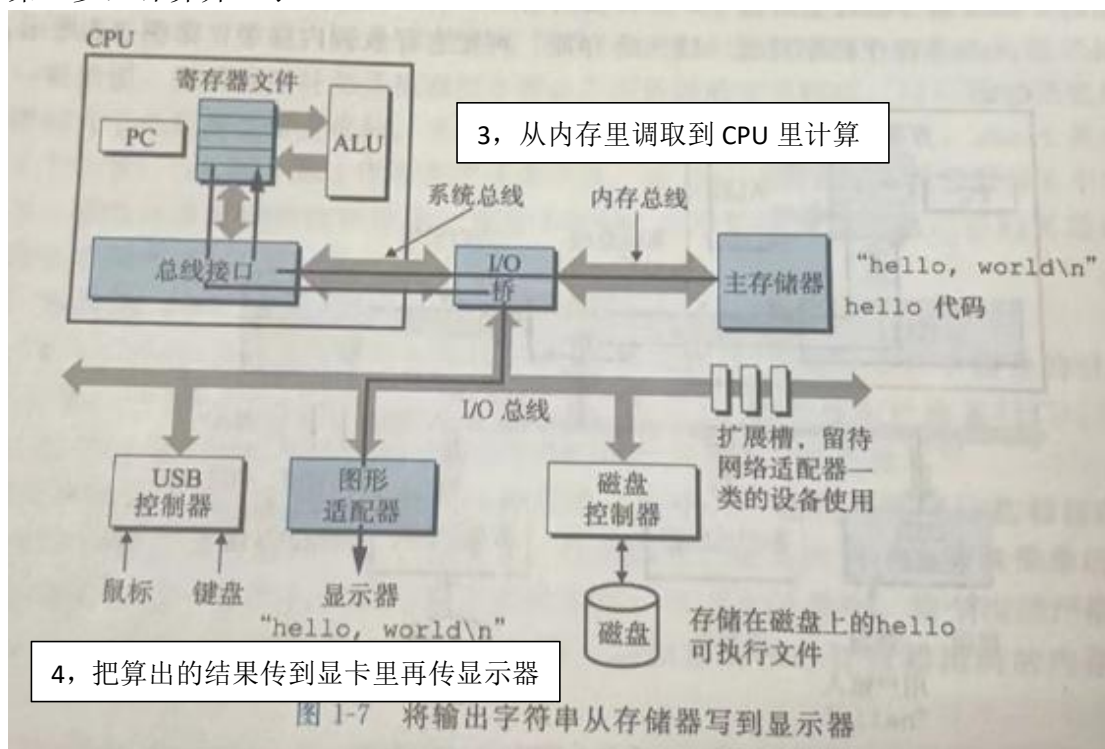
计算机操作系统是如何工作的，我们需要了解下

我们要执行一个已经保存在硬盘上的 hello world 的打印程序

第一步，加载程序



## 第二步，计算并显示



上述看似简单的过程，会引申出几个问题

问题 1，软件程序都存在硬盘上，如何设计运行规则，以保证流氓软件不乱来？

问题 2，内存条就这么大，我想边看小电影，边打游戏，一个游戏会同时运行很多代码，怎么保证高效使用内存，使得不卡顿？

问题 3，一个游戏的目录里包含了很多文件，如何管理与快速读取？

问题 4，我们的电脑用了很多年，小电影不够放就要加硬盘，鼠标影响我打游戏就要换鼠标，如何换了设备，电脑还能顺利运行，不产生额外问题？

以上问题都是操作系统来解决。

功能一，进程与线程（对应问题 1）

功能二，内存管理（对应问题 2）

功能三，文件管理（对应问题 3）

功能四，输入/输出（I/O）管理（对应问题 4）

### 计算机操作系统作用：

1，虚拟：为了驱动这堆电子配件，构建一个虚拟的操作环境，比如 win 系统；（对应问题 1，问题 4）

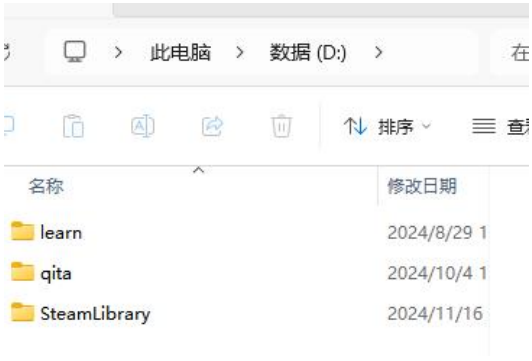
2，并行：让你能挂机游戏，边聊天，边看小电影；（对应问题 2）

3，持久：系统还能长久保存你的游戏进度，聊天记录，爱看的小电影；（对应问题 3）

### ★ 操作系统一般可以分为哪三个主要模块？

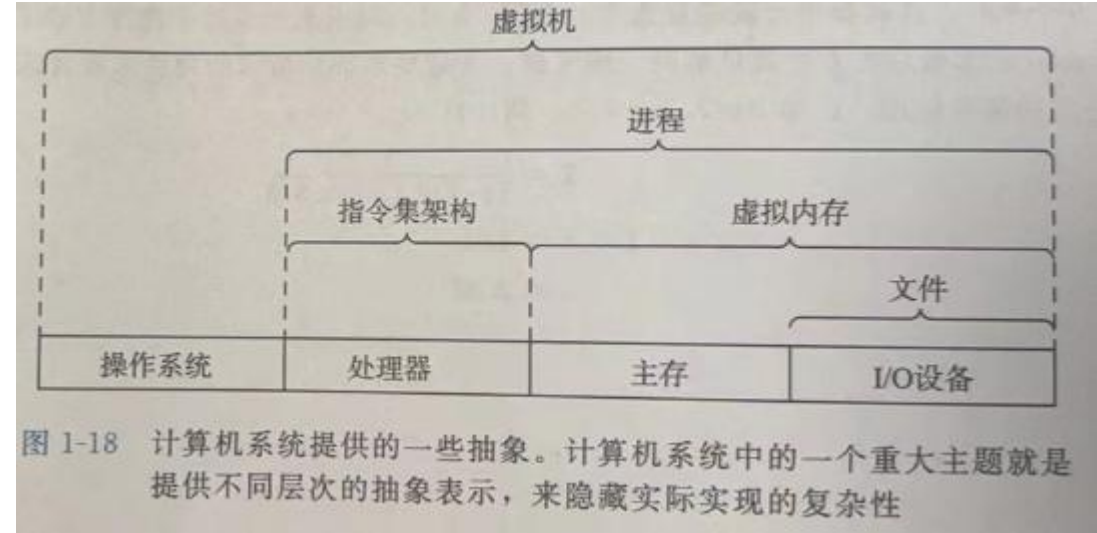
答：虚拟化（虚拟内存，进程），并行性，持久性（注：下表为相关名词解释）

|     |                                                                                            |                                                                                                                                                                                                                                                     |
|-----|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 模块  | 名词解释（★为考点）                                                                                 | 注释                                                                                                                                                                                                                                                  |
| 虚拟化 | <b>★进程</b><br>一个执行中程序的实例                                                                   |                                                                                                                                                                   |
|     | <b>★线程</b><br>运行在进程上下文中的逻辑流                                                                |  <p>（注：进程与线程区别，下载软件是一个实例，运行这个实例是进程，而实例可以多线程下载）</p>                                                                                                                |
|     | <b>★虚拟内存</b><br>硬件异常，硬件地址翻译、主存（即内存），磁盘（即硬盘）文件，内核软件（类win的桌面系统）的完美交互；<br>为进程提供大的，一致，私有的空间地址。 |  <p>（注：虚拟内存就干两件事，1，硬件与软件交互；2，为进程提供存放空间与地址）</p>                                                                                                                   |
| 并行性 | <b>★并行</b><br>两个逻辑控制流并发地运行在不同处理器上                                                          | <p><b>AI计算新里程碑：并行运算提升大模型训练效率</b></p> <p>2024-12-05 06:00</p> <p>在人工智能技术持续飞速发展的今天，AI大模型的广泛应用正面临一系列计算资源瓶颈问题。其中，如何高效地进行并行运算成为提升模型训练和推理效率的关键。随着技术进步，现代GPU（图形处理单元）已逐渐演变为高性能计算的重要动力，其在AI训练和推理中所扮演的角色愈发重要。</p> <p>（注：显卡上的GPU真并行，而CPU是单线程，多核CPU计算机也是并行）</p> |
|     | <b>★并发</b><br>一个逻辑控制流在时间上与另一个逻辑控制流重叠                                                       | <p>注：CPU是单线程，多个任务在一个CPU上交替运行，就是并发。</p> <p>比如打游戏时，聊天，电影可以同时进行，只因CPU算的快</p>                                                                                                                                                                           |

|     |                                                                                                                      |                                                                                    |
|-----|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 持久性 | <p>★文件系统</p> <p>存储设备上组织文件的方法。</p> <p>操作系统中负责管理和存储文件信息的软件称为文件管理系统，简称文件系统。</p> <p>三部分组成，文件系统接口，对象操控与管理的软件集合，对象和属性。</p> |  |
|-----|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|

★ 操作系统对上提供的抽象 (Abstract) 有什么优点？请给出 3 个或更多经典抽象的例子。

答：



- (注：I/O 设备就是显卡，显示器，键盘，鼠标，硬盘等)
- 1，操作系统
- 优点：-- 应用程序与硬件之间的软件，可防止硬件被失控的应用程序滥用；
- 面向应用程序提供简单一致的机制，由操作系统来控制复杂，大相径庭的低级硬件设备；
- 2，进程
- 优点：-- 一个独立的逻辑控制流，提供一个独占处理器的假象；
- 一个私有地址空间，提供一个独占使用内存的假象；
- 3，虚拟内存
- 优点：-- 硬盘与主存之间的高速缓存，使得主存上只保存活动区域，由虚拟内存依据需要在磁盘与主存之间来回传送数据，从而高效使用主存；
- 面向进程提供简单一致的地址空间，简化内存管理；
- 保护每个进程地址空间不被其它进程破坏；
- 4，文件
- (注：文件这条过于拗口，就不记了，反正有上面三条好记就满足题目要求了)

二，进程与线程

计算机从启动开始，到断电，这个过程中有一排程序在运行。  
如果正常情况，这些个进程，会以并行，或并发的方式，平滑的运行。

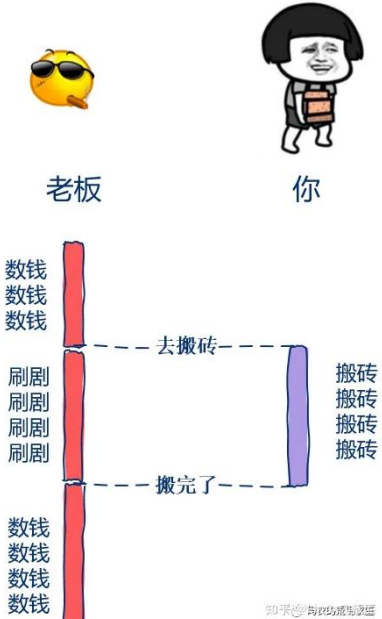
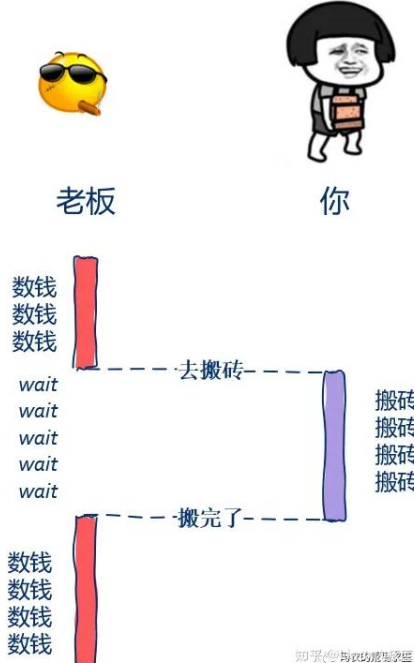
| 名词解释（★为考点）                                                              | 注释                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>★进程</div> <div>一个执行中程序的实例</div> <div>★pid=0</div> <div>指当前进程</div> | <div></div>                                                                                                                                                                                  |
| <div>★线程</div> <div>运行在进程上下文中的逻辑流</div>                                 | <div></div> <div>（注：进程与线程区别，<br/>下载软件是一个实例，运行这个实例是进程，而实例可以多线程下载）</div>                                                                                                                       |
| <div>★并行</div> <div>两个逻辑控制流并发地运行<br/>在不同处理器上</div>                      | <div><b>AI计算新里程碑：并行运算提升大模型训练效率</b></div> <div>2024-12-05 06:00</div> <div>在人工智能技术持续飞速发展的今天，AI大模型的广泛应用正面临一系列计算资源瓶颈问题。其中，如何高效地进行并行运算成为提升模型训练和推理效率的关键。随着技术进步，现代GPU（图形处理单元）已逐渐演变为高性能计算的重要动力，其在AI训练和推理中所扮演的角色愈发重要。</div> <div>（注：显卡上的 GPU 真并行，而 CPU 是单线程，<br/>多核 CPU 计算机也是并行）</div> |
| <div>★并发</div> <div>一个逻辑控制流在时间上与<br/>另一个逻辑控制流重叠</div>                   | <div>注：CPU 是单线程，多个任务在一个 CPU 上交替运行，就是并发。<br/>比如打游戏时，聊天，电影可以同时进行，只因 CPU 算的快</div>                                                                                                                                                                                                |

很多事情都是就怕但是，但是，人类就是有很多不满足，比如不想正常打游戏，就想开个挂啥的，还不想被抓到，同时还想不耽误下小电影，此时打游戏一定不能卡，不能卡，不能卡，否则影响我开挂的发挥，此时会遇到的问题及解决

★如何创建和初始化进程

建一个空的数据结构，然后填充每个域  
复制（比如 fork（）创建子进程，由父进程复制而来  
Exec（）传参，传进一个参数）

方案见下表：

| 名词解释（★为考点）                                                                                                          | 注释                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <p>★异常</p> <p>控制流中的突变，用来响应处理器状态中的某些变化</p>                                                                           |  <p>这样至少可以省电</p> |
| <p>★竞争</p> <p>当某程序多线程运行，其正确性是依赖一个线程达到控制流中的 y 点之前，另一个线程到达 x 点。如果程序猿没有按照规则规定，即多线程程序必须对任何可行的轨迹线都正确工作。此时该程序就出现竞争状态。</p> |                 |
| <p>★互斥</p> <p>想要确保每个线程在执行它的临界区中的指令时，拥有对共享变量的互斥访问。</p>                                                               |                |

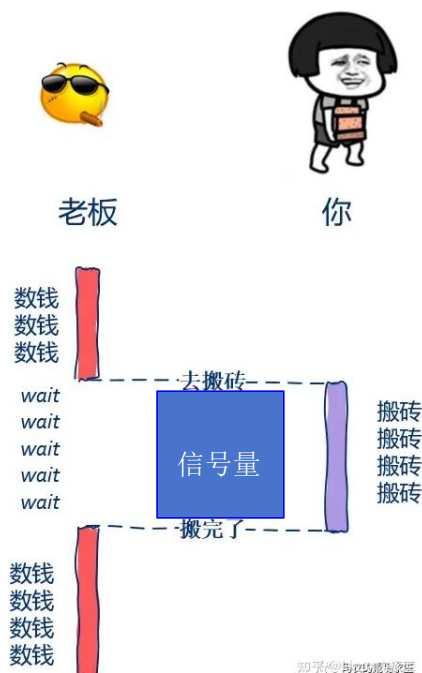
### ★信号量 (解决方案)

非负整数值的全局变量，只能有两种操作来处理 P 和 V，是解决不同线程，达成同步的方法。

(注：P 是获取资源，V 是释放资源，后续题目中会有详细解释)

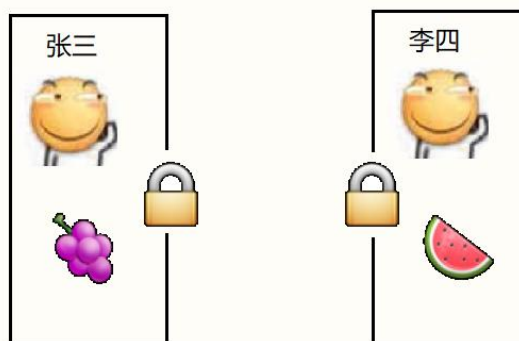
### ★信号

通知一个进程的一个消息



### ★死锁

一组线程被阻塞了，等一个永远不会为真的条件。



张三在吃葡萄，将葡萄和自己锁起来。

李四在吃西瓜，将自己和西瓜锁起来。

张三想吃西瓜，但是只能等待李四解锁不吃了然后给他

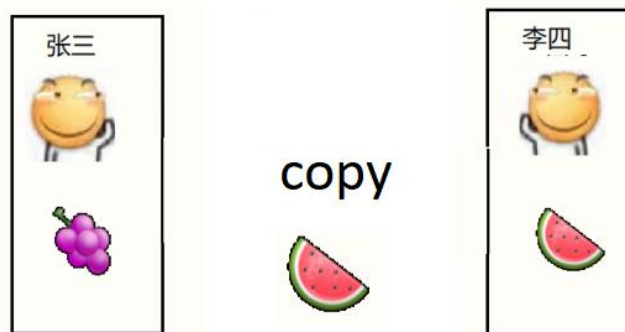
李四想吃葡萄，但是只能等待张三解锁不吃了然后给他

所以张三和李四都在等待对方不吃了，所以导致了“死锁”。

### ★无锁化

指多线程编程时，对线程间共享数据的并发修改不使用锁，采用基于硬件提供的原子操作能力来修改共享数据，进而提升性能，减少使用锁带来的开销。

(注：原子指不可分割，不会被打断)



原理就是拷贝一份数据，放外面

张三想吃西瓜

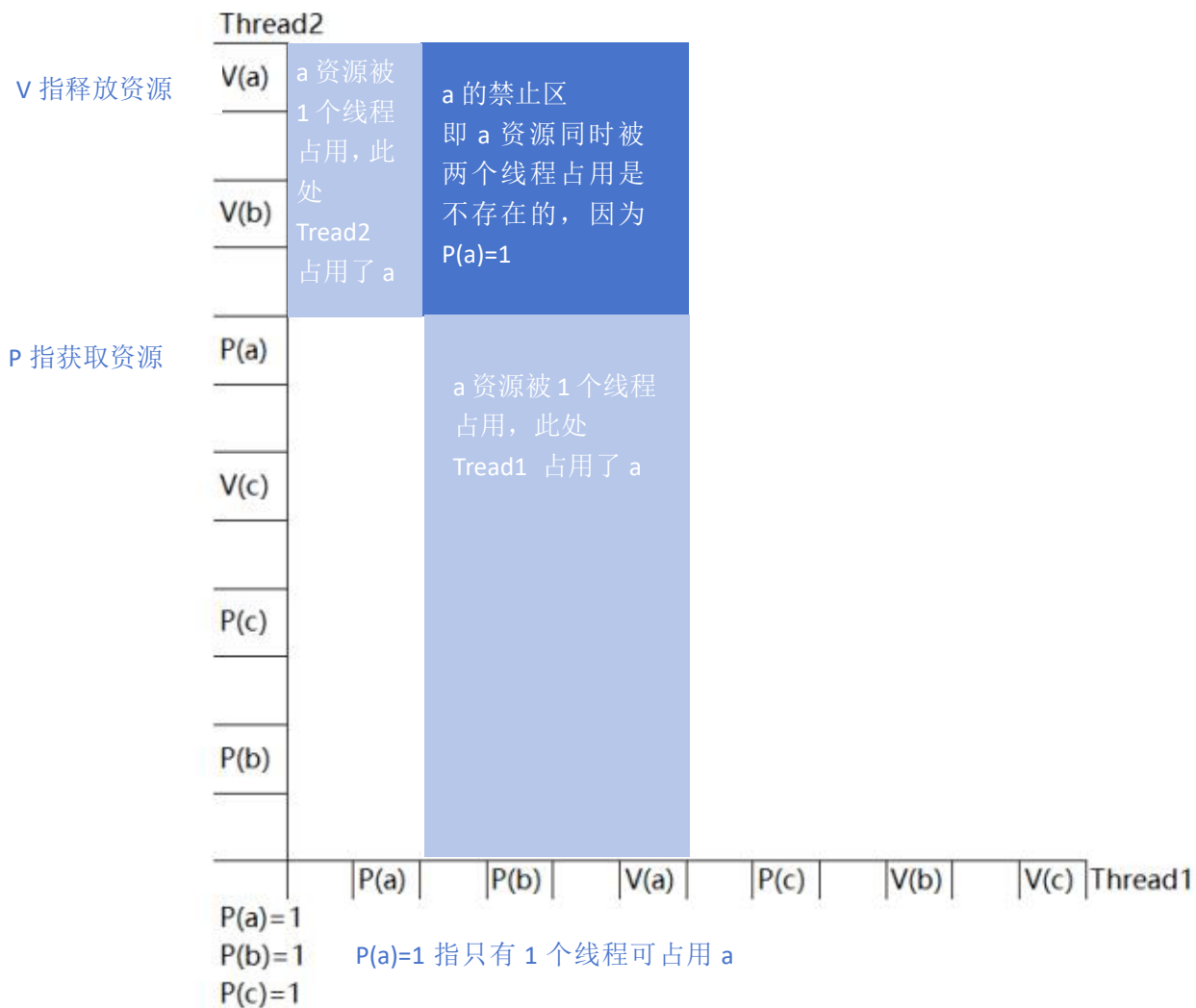
就看外面摆的是不是西瓜，如果是就拿来吃

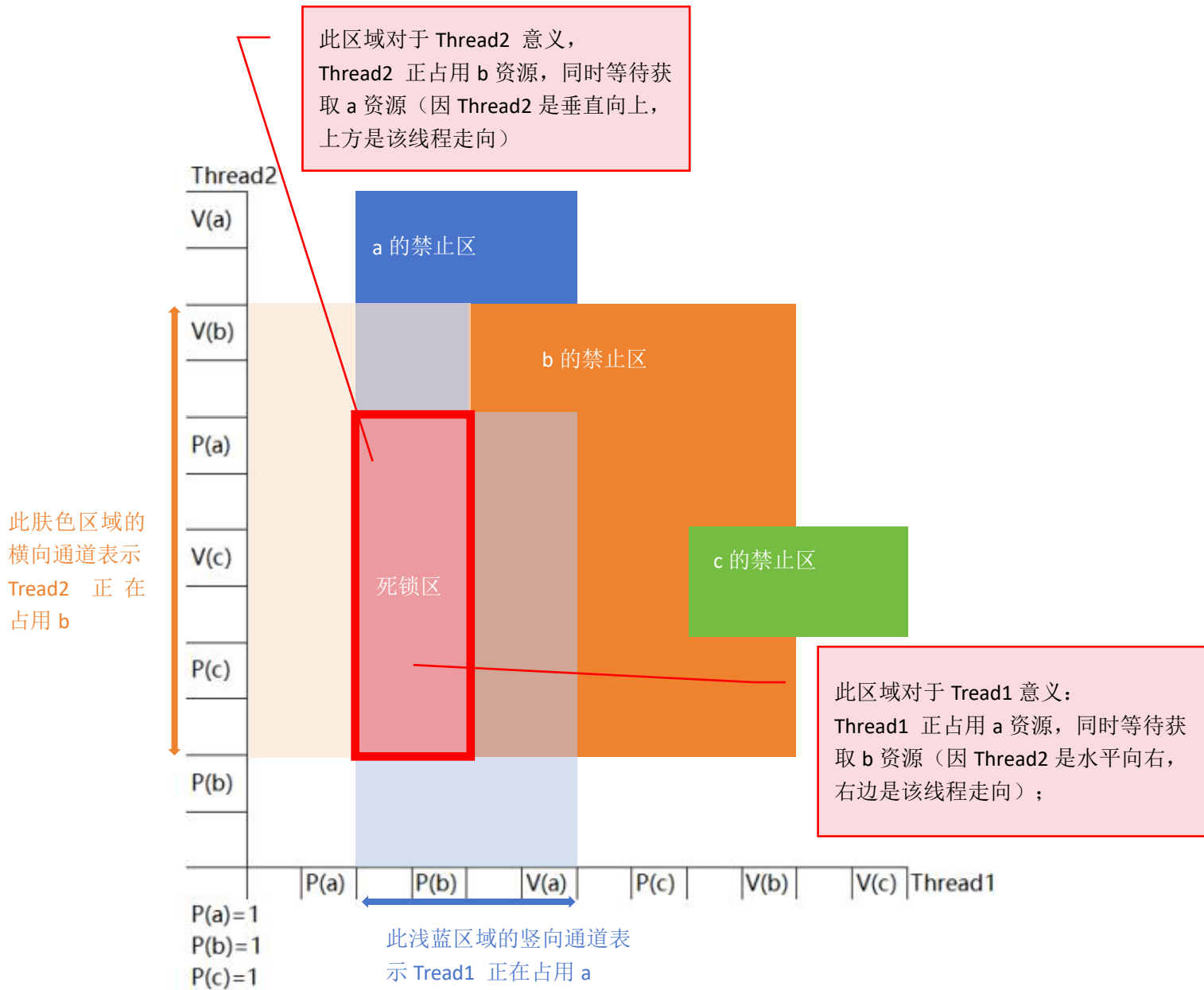
## ★ Deadlock 死锁

如下表所示，有两个线程 Thread1 和 Thread2 同时工作，a，b，c 为信号量，初始值均为 1，请问 Thread1 和 Thread2 是否会发生死锁？请画出进程图，并做出解释。

| Thread | Thread1 | Thread2 |
|--------|---------|---------|
| Step1  | P(a)    | P(b)    |
| Step2  | P(b)    | P(c)    |
| Step3  | V(a)    | V(c)    |
| Step4  | P(c)    | P(a)    |
| Step5  | V(b)    | V(b)    |
| Step6  | V(c)    | V(a)    |

（注：先看懂进度图，这里）



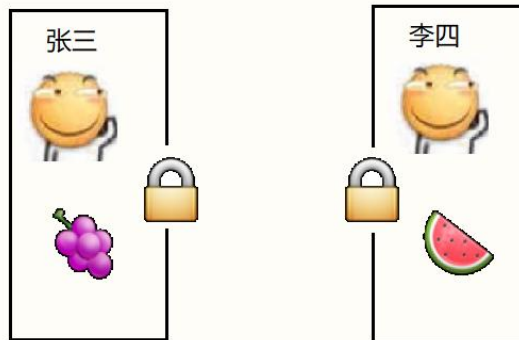


### ★死锁

一组线程被阻塞了，等一个永远不会为真的条件。

红框粉色区域：

Tread1 占用 a，  
Tread2 占用 b，  
Tread1 想要 b，只能等 Tread2 解锁后给 Tread1，  
Tread2 想要 a，只能等 Tread1 解锁后给 Tread2，  
所以 Tread1 和 Tread2 都在等对方解锁，所以导致死锁



张三在吃葡萄，将葡萄和自己锁起来。

李四在吃西瓜，将自己和西瓜锁起来。

张三想吃西瓜，但是只能等待李四解锁不吃了然后给他  
李四想吃葡萄，但是只能等待张三解锁不吃了然后给他  
所以张三和李四都在等待对方不吃了，所以导致了“死锁”。

答：会发生死锁，因为 a 的禁区与 b 的禁区的重叠区域引起一组死锁区域，会发生死锁。

## ★ 线程安全

请判断下面每个函数的类型是线程安全函数，线程不安全函数，还是可重入函数。

```
int t;

void f(int *x, int *y) {           //这是不安全函数，因为 t 为全局变量，如果多线程同时调用 f 函数，
    t = *x + *y;                   会造成同时修改 t 的值
}

void g(int*x, int *y) {             //这是不安全函数，因为 m 是静态局部变量，如多线程同时调用 g 函
    static int m = *x;              数，会造成竞争
    *y = *x;
    *x = m;
}

void h(int *x, int *y) {            //这是安全函数，因为 P 获取，V 释放的操作，使得其它线程调用
    P(&mutex);                      h 函数，因 t 变量被锁，不会被同时调用
    t = *x;
    *x = *y;
    *y = t;
    V(&mutex);
}

void k(int *x, int *y) {            //这是安全函数，因为 P 获取，V 释放有序进行，t 先锁，后释放
    P(&mutex);                      *y 再锁，后释放
    t = *x;
    V(&mutex);
    *x = *y;
    P(&mutex);
    *y = t;
    V(&mutex);
}

void m(int *x, int *y) {            //这是不安全函数，因为调取 m 函数时，内部变量都会由于外部线程
    int t = *x;                    同时调取 t，*x，*y 时，或中断，或改变值造成不一致
    *x = *y;
    *y = t;
}

void n(int *x, int *y) {            //x 与*x 有所不同，x 是变量指的在内存某个位置，存储的是地址
    *x = *x ^ *y;                  *x 指获取 x 所指向内存上这个变量指向的值
    *y = *x ^ *y;                  // *x = *x ^ *y 指 *x 保存了 *x，*y 指向的值，意味着外部线程
    *x = *x ^ *y;                  调用 n 时的影响，不会影响这个线程的后续操作，显然具有原子性
}                                   n 函数是可重入函数。可重入函数定义指在多线程环境中可以安全
被多任务调用的函数，不会被打断，影响，不会在运行过程中不断调用外部环境变量。

*y = *x ^ *y    // *y 保存了 *x 原始值；
*x = *x ^ *y    // *x 保存了 *y 原始值。
int a = 5; // 二进制表示为 0101
int b = 3; // 二进制表示为 0011
a = a ^ b; // a 现在是 0110 (6)
b = a ^ b; // b 现在是 0101 (5)
a = a ^ b; // a 现在是 0011 (3)
```

★ 下列程序会输出多少行 hello? 请画图展示 fork 和输出过程。

```
#include "csapp.h"
void doit() {
    printf("hello\n");
    fork();
    printf("hello\n");
    fork();
    printf("hello\n");
}

int main() {
    printf("hello\n");
    printf("hello\n");
    doit();
    printf("hello\n");
    exit(0);
}
```

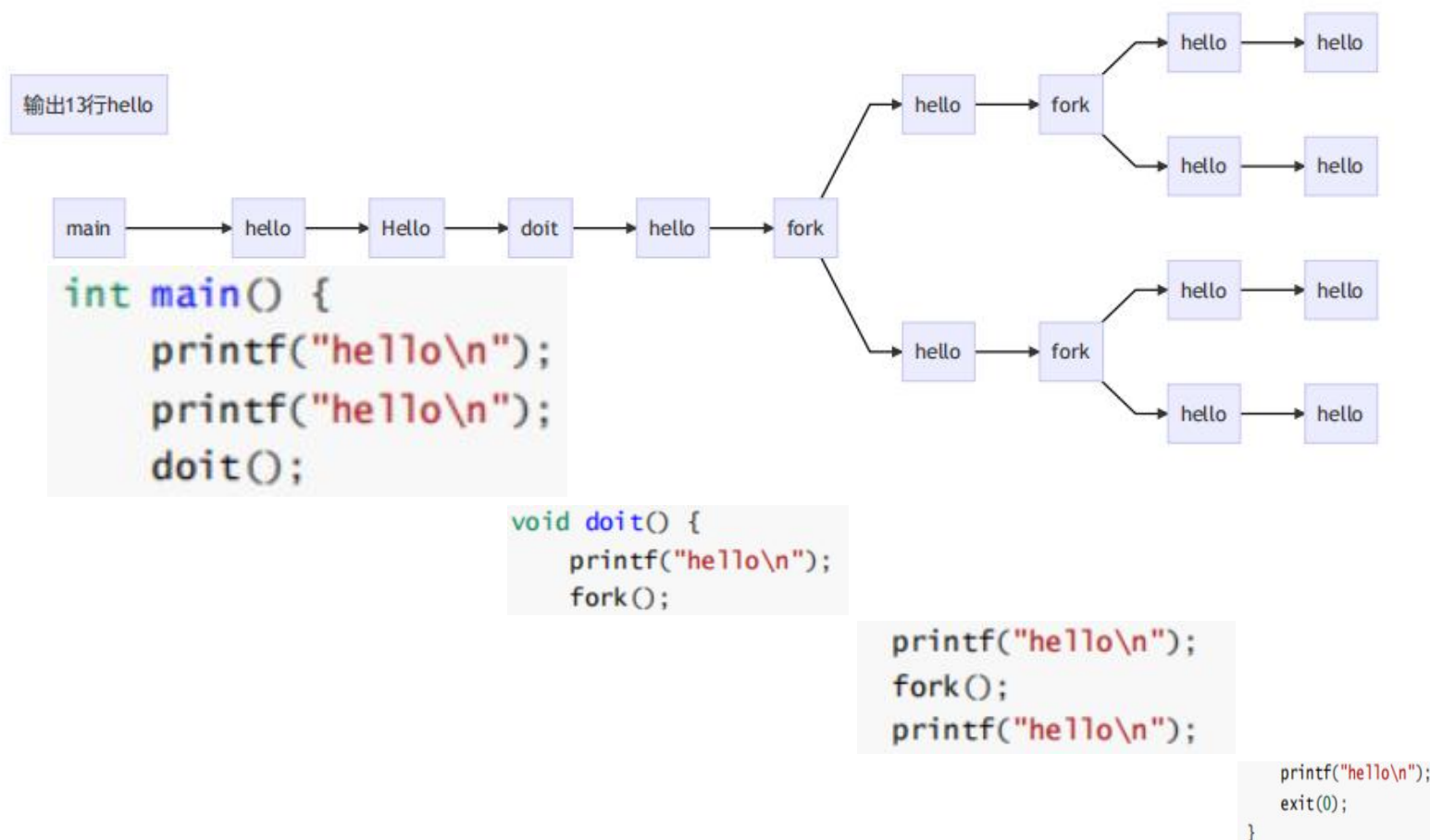
(注:

void doit () 表示自定义 doit 函数;

**fork ()** 表示创建子进程

int main () 表示程序起始)

答：



★ 下列程序会输出多少行 hello? 请画图展示 fork 和输出过程。

```
#include "csapp.h"
void doit() {
    printf("hello\n");
    fork();
    if(fork()==0)
        printf("hello\n");
    fork();
    printf("hello\n");
}

int main() {
    printf("hello\n");
    doit();
    exit(0);
}
```

此处应该是缩进  
If (fork () ==0)  
printf("hello\n")

(注:

void doit () 表示自定义 doit 函数;

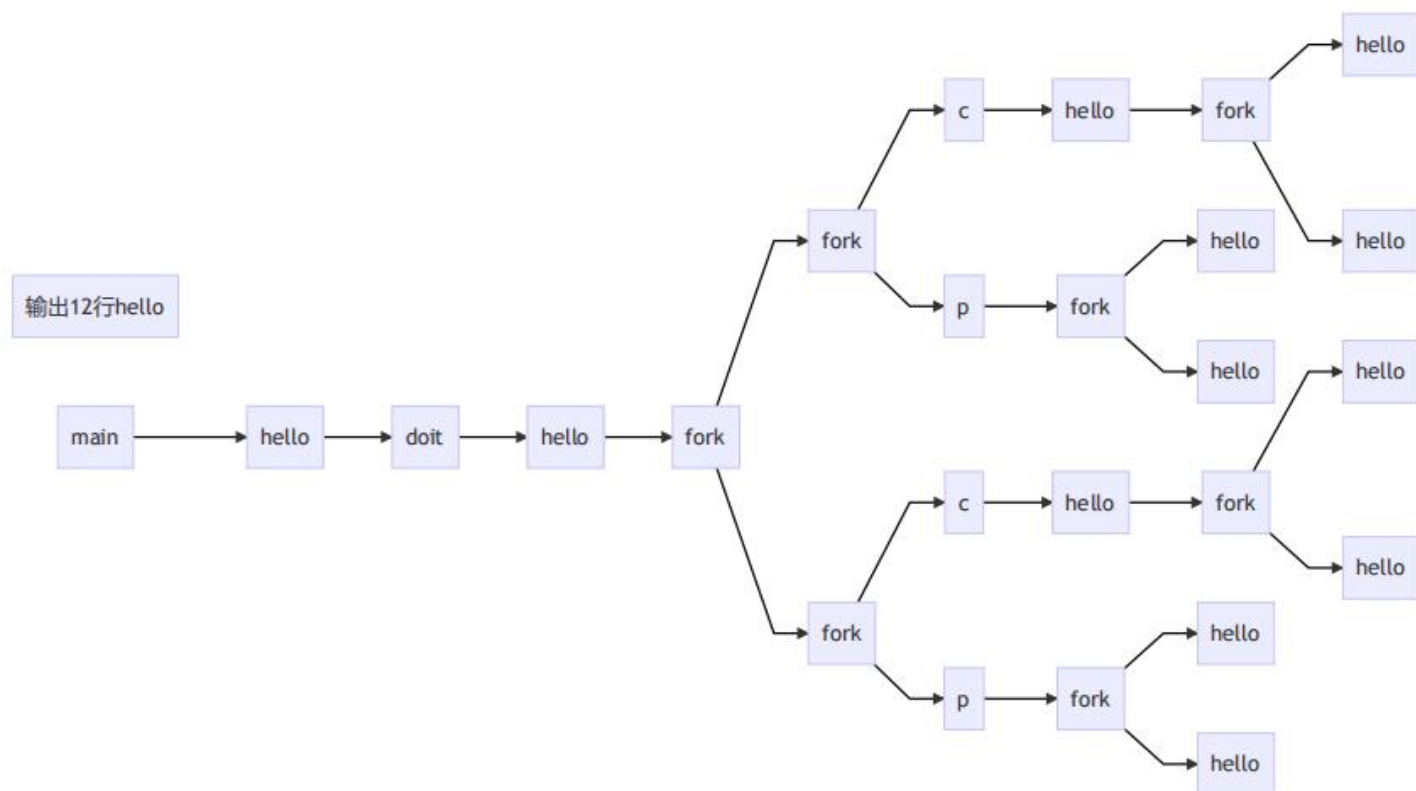
fork () 表示创建子进程

int main () 表示程序起始)

If (fork () ==0)

Print ("hello\n"); 表示如果是子进程就打印 hello, 父进程就执行下一步)

答:



★ 下列程序会输出多少行 hello? 请画图展示 fork 和输出的过程。

```
#include "csapp.h"
void doit() {
    for (int i=0;i<2;i++) {
        fork();
        printf("hello\n");
    }
    printf("hello\n");
}
int main() {
    printf("hello\n");
    doit();
    printf("hello\n");
    exit(0);
}
```

（注：

void doit（）表示自定义 doit 函数；

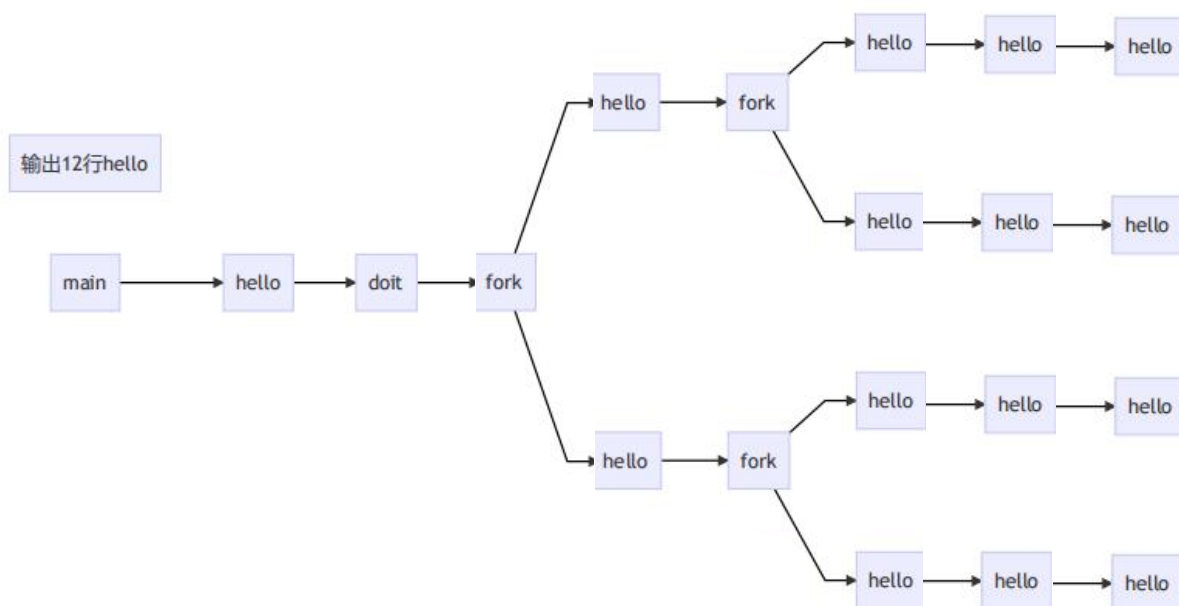
for（int=0;i<2;i++）表示循环执行 2 次

fork（）表示创建子进程

int main（）表示程序起始）

Print（"hello\n"）；表示如果是子进程就打印 hello，父进程就执行下一步）

答：输出 15 个 hello



## ★ 并发进程

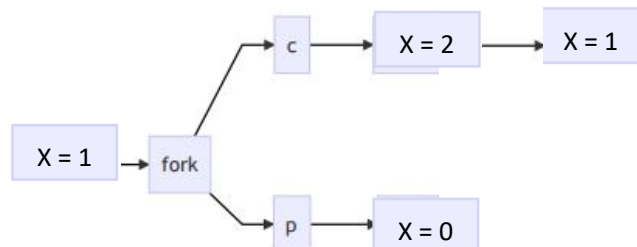
```
1  #include "csapp.h"
2
3  int main()
4  {
5      int x = 1;
6
7      if (Fork() == 0)
8          printf("printf1: x=%d\n", ++x);
9      printf("printf2: x=%d\n", --x);
10     exit(0);
11 }
```

问：子进程输出什么？主进程输出什么？共有多少种输出可能？

```
int main ()    //起始函数
{
    int = 1;    //设置 int 初始值为 1
    If (Fork () ==0) //如果是子进程
        print ( "printf1: x=%d\n" , ++x); //子进程打印 printf1: x=2 或者 1, ++x
        指 x+1, 如果
    printf( "printf2: x=%d\n" , --x); //父进程打印 printf2: x=0, --x 指 x-1
    Exit(0); //结束
}
```

但由于代码没有说明何时 fork 的子进程，因此存在以下三种情况

输出 201 或 210 或 021（注：两个线程，并发是交替进行，因此就会出现三种情况：）



201，那么 x=2 执行完后，执行 x=0，最后执行 x=1；

210，那么 x=2 执行完后，执行 x=1，最后执行 x=0；

021，那么先执行 x=0，再执行 x=2，最后执行 x=1；

总之 x=2 与 x=0 是并发，但 x=2 与 x=1 是按逻辑顺序进行。

## ★ 进程阻塞

进程阻塞原因不包括 A。

A 时间片切换 B 等待 I/O C 进程 sleep D 等待解锁

（注：进程三种状态：运行，准备，停止/阻塞。收到 SIG\*\*\*\*，一般都是进程停止，唯有收到 SIGCONT 进入运行状态）

## ★ CPU Scheduling

假定要在—台处理器上执行如下任务：0 时刻依次到达 1/2/3/4/5 任务，需要的执行时间分别是 10/3/5/1/8。给出采用下列调度算法时的调度顺序、平均周转时间和平均响应时间。

1. FCFS，任务不分时间片；
2. 非抢占式 SJF，任务不分时间片；
3. RR，时间片为 1，不考虑优先级

答：FCFS 用于进程调度算法，每次从队列中选择最先进入队列的进程，将 CPU 分配给它，使之投入运算，直到完成或阻塞才释放 CPU。

| 任务号 | 作业号 | 执行时间 | 开始时间 | 完成时间 | 周转时间 | 响应时间 |
|-----|-----|------|------|------|------|------|
| 1   | 1   | 10   | 0    | 10   | 10   | 0    |
| 2   | 2   | 3    | 10+0 | 10+3 | 13   | 10   |
| 3   | 3   | 5    | 3+10 | 13+5 | 18   | 13   |
| 4   | 4   | 1    | 5+13 | 18+1 | 19   | 18   |
| 5   | 5   | 8    | 1+18 | 19+8 | 27   | 19   |

平均周转时间=  $(10+13+18+19+27) / 5 = 17.4$

平均响应时间=  $(0+10+13+18+19) / 5 = 12$

调度顺序：1，2，3，4，5

SJF 用于进程调度算法，每次从队列中选择一个最短时间，将 CPU 分配给它，使之投入运算，直到完成或阻塞才释放 CPU。

| 任务号 | 作业号 | 执行时间 | 开始时间 | 完成时间  | 周转时间 | 响应时间 |
|-----|-----|------|------|-------|------|------|
| 1   | 5   | 10   | 9+8  | 17+10 | 27   | 17   |
| 2   | 2   | 3    | 0+1  | 1+3   | 4    | 1    |
| 3   | 3   | 5    | 1+3  | 4+5   | 9    | 4    |
| 4   | 1   | 1    | 0    | 1     | 1    | 0    |
| 5   | 4   | 8    | 4+5  | 9+8   | 17   | 9    |

平均周转时间=  $(27+4+9+1+17) / 5 = 11.6$

平均响应时间=  $(17+1+4+0+9) / 5 = 6.2$

调度顺序：4，2，3，5，1

RR 用于进程调度算法，任务排序，每隔 1 个时间片，序列头放序列尾，循环。

如下表数字为对应任务号

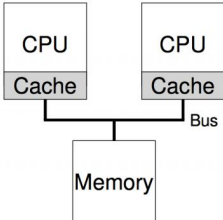
|     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 任务号 | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 5 | 1 | 2 | 3 | 5 | 1 | 3 | 5 | 1 | 3 | 5 | 1 | 5 | 1 | 5 | 1 | 5 | 1 | 1 |
| 时间片 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |

平均周转时间=  $(27+11+18+4+25) / 5 = 17$

平均响应时间=  $(0+1+2+3+4) / 5 = 2$

★ 多核 CPU 调度

此内容为老师直播课最后提到 CPU 的考核点，虽然没有习题，但会考这个概念。  
之前我们提到过如下知识点

|                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |       |   |   |   |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---|---|---|---|---|-------|---|---|-----|---|-----|-------|-------|---|---|---|---|-------|---|---|---|---|---|---|-----|-------|--|--|--|--|--|--|--|--|--|--|--|--|-------|---|---|---|---|---|---|---|---|---|---|---|---|
| <div>★并行</div> <div>两个逻辑控制流并发地运行在不同处理器上</div>                                                                                                                                                        | <div>AI计算新里程碑：并行运算提升大模型训练效率</div> <div>2024-12-05 06:00</div> <div>在人工智能技术持续飞速发展的今天，AI大模型的广泛应用正面临一系列计算资源瓶颈问题。其中，如何高效地进行并行运算成为提升模型训练和推理效率的关键。随着技术进步，现代GPU（图形处理单元）已逐渐演变为高性能计算的重要动力，其在AI训练和推理中所扮演的角色愈发重要。</div> <div>（注：显卡上的 GPU 真并行，而 CPU 是单线程，多核 CPU 计算机也是并行）</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |       |   |   |   |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>★并发</div> <div>一个逻辑控制流在时间上与另一个逻辑控制流重叠</div>                                                                                                                                                     | <div>注：CPU 是单线程，多个任务在一个 CPU 上交替运行，就是并发。</div> <div>比如打游戏时，聊天，电影可以同时进行，只因 CPU 算的快</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |       |   |   |   |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>问题：数据不一致。多核 CPU 问题是左图两个 CPU 调取同一份数据，一边改了，另一边没改，数据就会不一致。解决方案：总线监听（注：简单理解只认为最新数据为正确），这样会衍生一个问题：加锁消耗。防止死锁的性能消耗。解决方案：Cache Affinity 即缓存亲近。一个进程连续调度到同一个 CPU 核上，cache 和 TLB 都有效，性能更好。</div>           | <div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |       |   |   |   |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>基于以上有两种调度，一种是 Single-Queue Multiprocessor Scheduling (SQMS)</div> <div>优点：读取平衡即全局优化。</div> <div>缺点：锁开销大，缓存亲近差，性能差。</div>                                                                        | <div><table><tr><td>CPU 0</td><td>A</td><td>E</td><td>D</td><td>C</td><td>B</td></tr><tr><td>CPU 1</td><td>B</td><td>A</td><td>E</td><td>D</td><td>C</td></tr><tr><td>CPU 2</td><td>C</td><td>B</td><td>A</td><td>E</td><td>D</td></tr><tr><td>CPU 3</td><td>D</td><td>C</td><td>B</td><td>A</td><td>E</td></tr></table></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                          | CPU 0 | A | E | D | C | B | CPU 1 | B | A | E   | D | C   | CPU 2 | C     | B | A | E | D | CPU 3 | D | C | B | A | E |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 0                                                                                                                                                                                                | A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | E     | D | C | B |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 1                                                                                                                                                                                                | B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | A     | E | D | C |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 2                                                                                                                                                                                                | C                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | B     | A | E | D |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 3                                                                                                                                                                                                | D                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | C     | B | A | E |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>第二种，Multi-Queue Multiprocessor Scheduling (MQMS) 多队列多处理器调度策略。</div> <div>定义：每个 CPU 核一个独立的队列</div> <div>优点：性能好，死锁冲突少，缓存亲近好</div> <div>缺点：全局优化较差 load balance</div> <div>解决方案：work stealing</div> | <div><div>Q0 → A → C      Q1 → B → D</div><div><table><tr><td>CPU 0</td><td>A</td><td>A</td><td>C</td><td>C</td><td>A</td><td>A</td><td>C</td><td>C</td><td>A</td><td>A</td><td>C</td><td>C</td></tr><tr><td>CPU 1</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td></tr></table></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                | CPU 0 | A | A | C | C | A | A     | C | C | A   | A | C   | C     | CPU 1 | B | B | D | D | B     | B | D | D | B | B | D | D   |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 0                                                                                                                                                                                                | A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | A     | C | C | A | A | C | C     | A | A | C   | C |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 1                                                                                                                                                                                                | B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | B     | D | D | B | B | D | D     | B | B | D   | D |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>全局优化较差 load balance 指左图，A 执行完后就闲置了。</div>                                                                                                                                                       | <div><div>Q0 → A      Q1 → B → D</div><div><table><tr><td>CPU 0</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>A</td><td>...</td></tr><tr><td>CPU 1</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td><td>...</td></tr></table></div><div>Q0 →      Q1 → B → D</div><div><table><tr><td>CPU 0</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>CPU 1</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td><td>B</td><td>B</td><td>D</td><td>D</td></tr></table></div><div>Job C 执行完了，A 的份额是B,D 的两倍</div><div>Job A 也执行完了，CPU 0 完全闲置</div></div> | CPU 0 | A | A | A | A | A | A     | A | A | A   | A | ... | CPU 1 | B     | B | D | D | B | B     | D | D | B | B | D | D | ... | CPU 0 |  |  |  |  |  |  |  |  |  |  |  |  | CPU 1 | B | B | D | D | B | B | D | D | B | B | D | D |
| CPU 0                                                                                                                                                                                                | A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | A     | A | A | A | A | A | A     | A | A | ... |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 1                                                                                                                                                                                                | B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | B     | D | D | B | B | D | D     | B | B | D   | D | ... |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 0                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |       |   |   |   |   |   |       |   |   |     |   |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 1                                                                                                                                                                                                | B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | B     | D | D | B | B | D | D     | B | B | D   | D |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| <div>工作窃取 work stealing</div> <div>指不饱和队列从其它队列拿一两个任务但频率过高也会衍生其它问题。</div>                                                                                                                             | <div><table><tr><td>CPU 0</td><td>A</td><td>A</td><td>A</td><td>A</td><td>B</td><td>A</td><td>B</td><td>A</td><td>B</td><td>B</td><td>B</td><td>B</td></tr><tr><td>CPU 1</td><td>B</td><td>D</td><td>B</td><td>D</td><td>D</td><td>D</td><td>D</td><td>D</td><td>A</td><td>D</td><td>A</td><td>D</td></tr></table></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                | CPU 0 | A | A | A | A | B | A     | B | A | B   | B | B   | B     | CPU 1 | B | D | B | D | D     | D | D | D | A | D | A | D   |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 0                                                                                                                                                                                                | A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | A     | A | A | B | A | B | A     | B | B | B   | B |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |
| CPU 1                                                                                                                                                                                                | B                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | D     | B | D | D | D | D | D     | A | D | A   | D |     |       |       |   |   |   |   |       |   |   |   |   |   |   |     |       |  |  |  |  |  |  |  |  |  |  |  |  |       |   |   |   |   |   |   |   |   |   |   |   |   |

## ★ Signal 信号

下列程序的输出是？

```
pid_t pid;                //声明一个 pid 变量
int counter = 2;          //全局变量 counter 为 2
void handler1(int sig) {  //定义信号处理函数 handler1
    counter = counter - 1; //全局变量自减 1
    printf("%d", counter); //打印 counter 值
    fflush(stdout);        //确保上述操作立即执行
    exit(0);              //结束进程
}

int main() {              //起始函数
    signal(SIGUSR1, handler1); //设置 SIGUSR1 信号的处理函数 handler1
    printf("%d", counter);    //打印 counter 初始值 2
    fflush(stdout);          //确保上述操作立即执行
    if ((pid = fork()) == 0) { //创建一个子进程，如果当前是子进程
        while(1) {};          //子进程无限循环操作，即什么也不做
    }

    kill(pid, SIGUSR1);       //父进程向子进程发送 SIGUSR1 信号
    waitpid(-1, NULL, 0);     //父进程等待子进程结束
    counter = counter + 1;    //counter 自加 1
    printf("%d", counter);    //打印 counter 的值
    exit(0);                 //结束进程
}
```

整个过程即

- 1, 定义一个叫 SIGUSR1 的信号可发向 handler1;
- 2, 打印初始值 2;
- 3, 创建子进程，子进程等待;
- 4, 父进程发信号 SIGUSR1，即子进程执行 handler1 函数;
- 5, 打印 1，结束子进程
- 6, 父进程等到了子进程结束，于是继续执行父进程
- 7, Counter 自加 1，即  $2+1=3$
- 8, 打印 3
- 9, 结束进程

所以最终输出 213

### 三，内存管理

内存管理很像物流仓储集散中心，货物（数据）经过物流管理（内存管理），暂时存在仓库（内存），货物有物流编码（数据逻辑地址），也有对于仓储的货物仓储号（数据物理地址），物流管理（内存管理）会在物流编码（数据逻辑地址）与仓储号（数据物理地址）间做转换（地址翻译）



但公司资源饱和（内存小）但还要发展（任务多）还想给牛马们点福报（更多任务），于是设计了 996 给员工（虚拟内存）

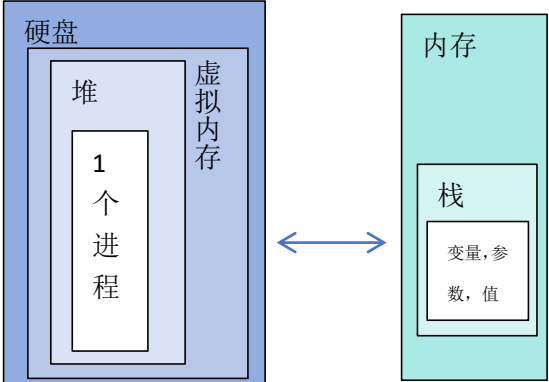
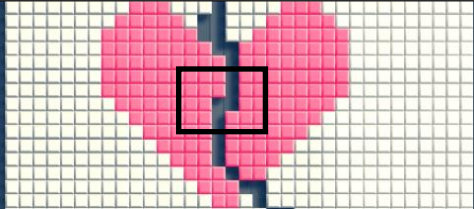
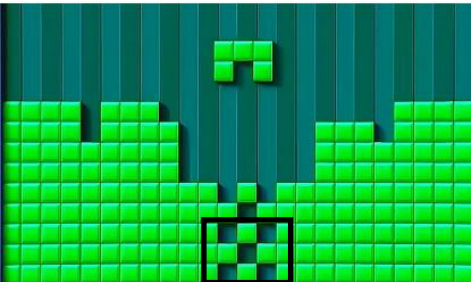


实际大多数业务（任务）都在员工（虚拟内存）手里，而正在挣钱的业务（活跃任务）在领导（内存）手里，领导的汇报总显得卓有成效（高效），而我的汇报总是充满着工作量（任务多），这就是领导（内存）与我（虚拟内存）的区别。

现在就有个问题，为啥我的工作显得没有成效，原因如下：

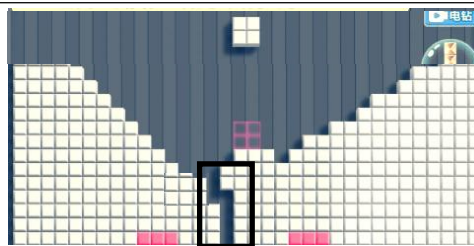
1，我的工作都是临时（动态分配器）分配的零碎活（堆 heap）；

- 2, 这堆工作是如何分配的, 分配规则由领导决定 (malloc 函数分配)  
分配标准就三条, 一是工作饱和 (最大吞吐率), 二是充分被利用, 最好能背锅 (最大化内存利用率), 三是和领导关系 (关系指吞吐率与利用率的算法平衡);
- 3, 我们公司是家大公司, 部门协同很常见, 但也造成了人工效率逐渐低下, 往往等待时间 (碎片) 比干活的时间还长, 我经常电脑开着让它自己跑测试, 然后躲哪里划划水 (内部碎片), 有的活耗时长还没法拆分, 我手头杂活排的很满, 零碎时间不够搞 (外部碎片)。

| 名词解释 (★为考点)                                                                                            | 注释                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>★虚拟内存</b><br>硬件异常, 硬件地址翻译、主存 (即内存), 磁盘 (即硬盘) 文件, 内核软件 (类 win 的桌面系统) 的完美交互;<br>为进程提供大的, 一致, 私有的空间地址。 |  <p>(注: 虚拟内存就干两件事, 1, 硬件与软件交互; 2, 为进程提供存放空间与地址)</p> |
| <b>★栈 (内存上)</b><br>编译器自动维护着一个线程的固定大小的内存区域, 称为栈 (程序员无法编译, 随用随清理, 无碎片)                                   |                                                   |
| <b>★堆 (虚拟内存上)</b><br>动态内存分配器维护着一个进程的虚拟内存区域, 称为堆。 (需要程序员申请分配与释放)<br>(注: 教材上堆在虚拟内存里, 网上说在内存里, 以教材为准)     |                                                                                                                                       |
| <b>★碎片 (堆才有碎片)</b><br>堆里的有未被使用的内存但不能用来满足分配请求, 这些不能分配的块叫碎片。                                             |                                                   |
| <b>★内部碎片</b><br>处于已分配区域内部的存储块, 这区域被进程占有, 同时占有的这些块无法被系统利用, 直到进程释放或结束, 系统才可能利用这个存储块。                     |                                                   |

### ★外部碎片

处于未分配区域的储存块，这些存储块总和可以满足当前申请的长度要求，但他们不连续或其它原因，无法满足当前申请。



### ★ malloc 内存分配需要考虑那两个指标？它们之间是什么关系？

答：1，最大吞吐率：

吞吐率定义为每个单位时间里完成的请求数。如一个分配器 1 秒内完成 500 个分配请求和 500 个释放请求，那么它的吞吐率是 1000 次每秒操作。

2，最大化内存利用率：

所有进程分配的虚拟内存的全部数量受磁盘上可交换空间数量的限制，虚拟内存是一个有限空间，必须高效使用。

3，关系：

最大化吞吐率与最大化内存利用率之间是互相牵制关系。以堆利用率为代价，很容易编写出吞吐率最大化的分配器。分配器设计中一个有趣挑战就是这两个目标之间找一个适当的平衡。

其实以上还不是最要命的，最要吐槽的是公司制度。大老板（CPU）想了个愿景（虚拟地址），把愿景指派给总监（内存存储单元 MMU），总监也没有把这事儿丢给我的领导，而是暗示（PTE 地址，即 PTEA），领导（高速缓存/主存）是个机灵人主动向总监申请了个活（PTE），给到总监意思是团队请求干这活（PTE），总监就发了邮件（物理地址）给我的领导，批准了。领导最后给了老板（CPU）一个漂亮的 PPT（数据字），其它干没干成，取决于我。

### ★ 操作系统的虚拟内存机制中，软硬件是如何配合工作的？请简述。

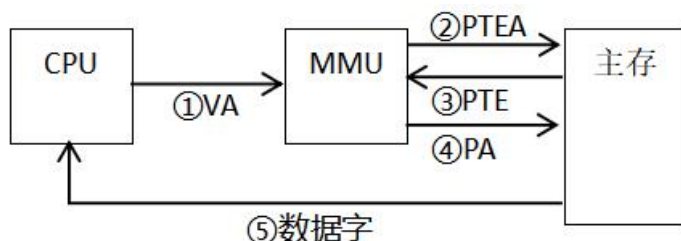
答：1，CPU 生成一个虚拟地址（VA），并把它传给芯片上的 MMU

2，MMU 生成一个 PTE 地址（PTEA），并向主存请求这个 PTE

3，主存向 MMU 返回 PTE

4，MMU 构造物理地址（PA），并发给主存

5，主存返回 CPU 数据字



★ Address Translation 地址翻译

1，一台计算机中的内存访问需要先将虚拟地址转为物理地址，下面是该计算机系统进行地址转换时相关的一些参数：

- 主存的每个 byte 都有一个独立的地址；
- 系统采用单层页表；
- 主存每次访问的粒度是一个字节，该计算机的 word 定位为 1Byte；
- 虚拟地址由 25bits 组成；
- TLB 是 4 路组相连，一共能存储 16 个 entry；
- PPN（Physical Page Number）是 9bits；
- PTE 的总数量是 $2^{14}$ ；

请填写下表：

|                      |  |
|----------------------|--|
| VP0 中包含的 bit 数量      |  |
| 物理地址的 bit 数          |  |
| 页大小                  |  |
| TLBT（TLB Tag）的 bit 数 |  |

答：请看下表它们数量对应关系，并记住如下公式(注：以下地址为二进制)

|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---------------------------------------------------------|---|---|---|---|---|---|---|---|--|--|--|--|
| VA:0000001001111001011010011 |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
| VPN                          |   |   |   |   |   |   |   |   |   |   |   |   | VPO                                                     |   |   |   |   |   |   |   |   |  |  |  |  |
| 0                            | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 0                                                       | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |  |  |  |  |
| TLBT                         |   |   |   |   |   |   |   |   |   |   |   |   | TLBI                                                    |   |   |   |   |   |   |   |   |  |  |  |  |
| 关系                           |   |   |   |   |   |   |   |   |   |   |   |   | <div><div>TLB</div><div>PTE</div><div>页 (P)</div></div> |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |
|                              |   |   |   |   |   |   |   |   |   |   |   |   |                                                         |   |   |   |   |   |   |   |   |  |  |  |  |

## 2, 如下 TLB, PTE 表

TLB 中的内容如下 (数字为 16 进制) :

| Set | Tag | PPN | Valid | Tag | PPN | Valid |
|-----|-----|-----|-------|-----|-----|-------|
| 0   | 0CD | 09  | 1     | AA1 | 00  | 1     |
|     | 3E0 | 62  | 0     | C4C | 48  | 1     |
| 1   | 312 | 45  | 0     | 010 | 75  | 1     |
|     | 987 | 3A  | 1     | D39 | 3F  | 0     |
| 2   | 038 | E3  | 0     | 0A7 | 13  | 1     |
|     | 18B | 52  | 1     | 49B | 11  | 0     |
| 3   | 6C0 | 42  | 0     | 075 | 50  | 0     |
|     | 013 | 39  | 1     | 0F2 | 0D  | 0     |

TLB: 4 sets, 16 entries, 4-way set associative

Page Table 中前 16 个 entry 的内容如下 (数字为 16 进制) :

| VPN | PPN | Valid | VPN | PPN | Valid |
|-----|-----|-------|-----|-----|-------|
| 00  | 39  | 1     | 08  | 0D  | 0     |
| 01  | 52  | 1     | 09  | 45  | 0     |
| 02  | E3  | 0     | 0A  | 13  | 1     |
| 03  | 00  | 1     | 0B  | 3A  | 1     |
| 04  | 11  | 0     | 0C  | 09  | 1     |
| 05  | 50  | 0     | 0D  | 42  | 0     |
| 06  | 62  | 0     | 0E  | 3F  | 0     |
| 07  | 75  | 1     | 0F  | 48  | 1     |

Page Table: Only the first 16 PTEs are shown

请将以下虚拟地址转为物理地址并填写以下表格

| Parameter         | Value     | Value     |
|-------------------|-----------|-----------|
| VA                | 0x014F2D3 | 0x07C01A5 |
| VPN               |           |           |
| TLB index (TLBI)  |           |           |
| TLB Tag (TLBT)    |           |           |
| TLB Hit? (Y/N)    |           |           |
| Page Fault? (Y/N) |           |           |
| PPN               |           |           |
| PA                |           |           |

答:

VA 地址是十六进制, 需要转二进制才能进行后续运算, 转表如下:

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

①VA 已给 0x014F2D3, (注: 0x 为固定开头范式, 不用管)

|    |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 4    | F    | 2    | D    | 3    |
| 二  | 0000 | 0001 | 0100 | 1111 | 0010 | 1101 | 0011 |

②回到公式  $VA_n = VPN + VPO$

上一小题  $n = 25$ ,  $VPN = 14$ ,  $VPO = 11$ , 得如下表

|    |          |      |      |      |          |      |      |
|----|----------|------|------|------|----------|------|------|
| 十六 | 0        | 1    | 4    | F    | 2        | D    | 3    |
| 二  | 0000     | 0001 | 0100 | 1111 | 0010     | 1101 | 0011 |
| VA |          |      |      |      | 010      | 1101 | 0011 |
|    |          |      |      |      | 11 位 VPO |      |      |
|    | 0        | 0001 | 0100 | 1111 | 0        |      |      |
|    | 14 位 VPN |      |      |      |          |      |      |

因 VPN 转十六位需要满足 16 位, 因此补 2 位 00 即

|      |      |      |      |
|------|------|------|------|
| 0000 | 0010 | 1001 | 1110 |
|------|------|------|------|

对照十六转二表:

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

得十六位  $VPN = 0x029E$

|    |      |      |      |      |
|----|------|------|------|------|
| 十六 | 0    | 2    | 9    | E    |
| 二  | 0000 | 0010 | 1001 | 1110 |

③回到公式  $VPN = TLBT + TLBI$

上一小题  $VPN = 14$ ,  $TLBT = 12$ ,  $TLBI = 2$ ,

|    |           |      |      |          |
|----|-----------|------|------|----------|
| 十六 | 0         | 2    | 9    | E        |
| 二  | 0000      | 0010 | 1001 | 1110     |
|    |           |      |      | 10       |
|    |           |      |      | 2 位 TLBI |
|    | 00        | 0010 | 1001 | 11       |
|    | 12 位 TLBT |      |      |          |

TLBI 为转十六需补 00 达成 4 位

|      |
|------|
| 2    |
| 0010 |

对照十六转二表:

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

十六位  $TLBI = 0x2$

|      |      |      |
|------|------|------|
| 0    | A    | 7    |
| 0000 | 1010 | 0111 |

十六位  $TLBT = 0x0A7$

#### ④通过查表，获取 PPN

TLB 中的内容如下（数字为 16 进制）：

| Set | Tag | PPN | Valid | Tag | PPN | Valid |
|-----|-----|-----|-------|-----|-----|-------|
| 0   | 0CD | 09  | 1     | AA1 | 00  | 1     |
|     | 3E0 | 62  | 0     | C4C | 48  | 1     |
| 1   | 312 | 45  | 0     | 010 | 75  | 1     |
|     | 987 | 3A  | 1     | D39 | 3F  | 0     |
| 2   | 038 | E3  | 0     | 0A7 | 13  | 1     |
|     | 18B | 52  | 1     | 49B | 11  | 0     |
| 3   | 6C0 | 42  | 0     | 075 | 50  | 0     |
|     | 013 | 39  | 1     | 0F2 | 0D  | 0     |

TLB: 4 sets, 16 entries, 4-way set associative

Page Table 中前 16 个 entry 的内容如下（数字为 16 进制）：

| VPN | PPN | Valid | VPN | PPN | Valid |
|-----|-----|-------|-----|-----|-------|
| 00  | 39  | 1     | 08  | 0D  | 0     |
| 01  | 52  | 1     | 09  | 45  | 0     |
| 02  | E3  | 0     | 0A  | 13  | 1     |
| 03  | 00  | 1     | 0B  | 3A  | 1     |
| 04  | 11  | 0     | 0C  | 09  | 1     |
| 05  | 50  | 0     | 0D  | 42  | 0     |
| 06  | 62  | 0     | 0E  | 3F  | 0     |
| 07  | 75  | 1     | 0F  | 48  | 1     |

Page Table: Only the first 16 PTEs are shown

TLBI = 0x2

TLBT = 0x0A7

通过查表，TLB 表中 Set=2，Tag 中存在 0A7，所以 TLB Hit 为 Y

在表 Page Table 中存在 PPN=13 且 Valid= 1，所以 Page Fault 为 N 表示不缺页

（注：TLB 理解为一张可以快速检索 PTE 的映射表，可以快速获取 PPN）

对照十六转二表：

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

PPN = 0x13 的二进制是

|      |      |
|------|------|
| 1    | 3    |
| 0001 | 0011 |

#### ⑤步骤②得

|          |      |      |
|----------|------|------|
| 010      | 1101 | 0011 |
| 11 位 VPO |      |      |

回到公式，PPO = VPO，物理地址 m = PPN + PPO，题干 PPN=9，得

|                          |      |      |          |      |      |
|--------------------------|------|------|----------|------|------|
| 0                        | 0001 | 0011 | 010      | 1101 | 0011 |
| 9 位 PPN（13 只有 8 位需要前补 0） |      |      | 11 位 PPO |      |      |

最后整理得 PA = 0x09AD3

|    |      |      |      |      |      |
|----|------|------|------|------|------|
| 十六 | 0    | 9    | A    | D    | 3    |
| 二  | 0000 | 1001 | 1010 | 1101 | 0011 |

综上得

| Parameter         | Value     |
|-------------------|-----------|
| VA                | 0x014F2D3 |
| VPN               | 0x029E    |
| TLB index (TLBI)  | 0x2       |
| TLB Tag (TLBT)    | 0x0A7     |
| TLB Hit? (Y/N)    | Y         |
| Page Fault? (Y/N) | N         |
| PPN               | 0x13      |
| PA                | 0x09AD3   |

第二个 VA 已给 0x07C01A5（注：0x 为固定开头范式，不用管）  
VA 地址是十六进制，需要转二进制才能进行后续运算，转表如下：

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

①得 VA 二进制

|    |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|
| 十六 | 0    | 7    | C    | 0    | 1    | A    | 5    |
| 二  | 0000 | 0111 | 1100 | 0000 | 0001 | 1010 | 0101 |

②回到公式  $VA_n = VPN + VPO$

上一题已知， $n = 25$ ， $VPN = 14$ ， $VPO = 11$ ，得

|    |      |          |      |      |          |      |      |
|----|------|----------|------|------|----------|------|------|
| 十六 | 0    | 7        | C    | 0    | 1        | A    | 5    |
| 二  | 0000 | 0111     | 1100 | 0000 | 0001     | 1010 | 0101 |
| VA |      |          |      |      | 001      | 1010 | 0101 |
|    |      |          |      |      | 11 位 VPO |      |      |
|    | 0    | 0111     | 1100 | 0000 | 0        |      |      |
|    |      | 14 位 VPN |      |      |          |      |      |

14 位二进制 VPN 需前加 00 补齐 16 位，再转十六进制，得

|    |      |      |      |      |
|----|------|------|------|------|
| 十六 | 0    | F    | 8    | 0    |
| 二  | 0000 | 1111 | 1000 | 0000 |

$VPN = 0x0F80$

③回到公式  $VPN = TLBT + TLBI$

上一题已知， $TLBT = 12$ ， $TLBI = 2$ ，得  $TLBI = 0x0$ （注：补 00，满 4 位才能转十六）

$TLBT = 0011\ 1110\ 0000 = 0x3E0$

|    |           |      |      |          |
|----|-----------|------|------|----------|
| 十六 | 1         | F    | 0    | 0        |
| 二  | 0000      | 1111 | 1000 | 0000     |
|    |           |      |      | 00       |
|    |           |      |      | 2 位 TLBI |
|    | 00        | 1111 | 1000 | 00       |
|    | 12 位 TLBT |      |      |          |

④通过查表

TLB 中的内容如下（数字为 16 进制）：

| Set | Tag | PPN | Valid | Tag | PPN | Valid |
|-----|-----|-----|-------|-----|-----|-------|
| 0   | 0CD | 09  | 1     | AA1 | 00  | 1     |
|     | 3E0 | 62  | 0     | C4C | 48  | 1     |
| 1   | 312 | 45  | 0     | 010 | 75  | 1     |
|     | 987 | 3A  | 1     | D39 | 3F  | 0     |
| 2   | 038 | E3  | 0     | 0A7 | 13  | 1     |
|     | 18B | 52  | 1     | 49B | 11  | 0     |
| 3   | 6C0 | 42  | 0     | 075 | 50  | 0     |
|     | 013 | 39  | 1     | 0F2 | 0D  | 0     |

TLB: 4 sets, 16 entries, 4-way set associative

Page Table 中前 16 个 entry 的内容如下（数字为 16 进制）：

| VPN | PPN | Valid | VPN | PPN | Valid |
|-----|-----|-------|-----|-----|-------|
| 00  | 39  | 1     | 08  | 0D  | 0     |
| 01  | 52  | 1     | 09  | 45  | 0     |
| 02  | E3  | 0     | 0A  | 13  | 1     |
| 03  | 00  | 1     | 0B  | 3A  | 1     |
| 04  | 11  | 0     | 0C  | 09  | 1     |
| 05  | 50  | 0     | 0D  | 42  | 0     |
| 06  | 62  | 0     | 0E  | 3F  | 0     |
| 07  | 75  | 1     | 0F  | 48  | 1     |

Page Table: Only the first 16 PTEs are shown

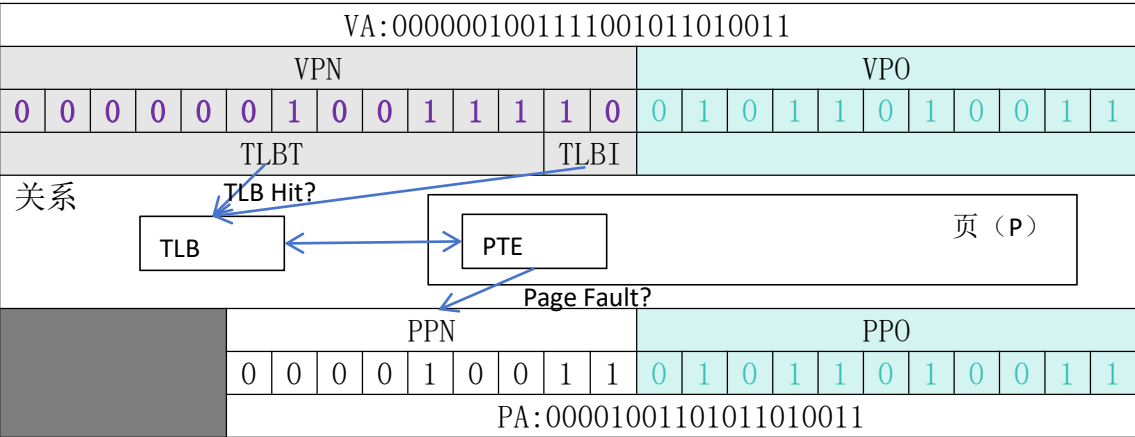
TLBI = 0x0, TLBT = 0x3E0

TLB 表中查得 Set 0 中 3E0 存在，说明 TLB Hit 为 Y

但 Page Table 表中 PPN = 62，但 Valid = 0，说明 Page Fault 为 Y，表示该条目当前无效，为缺省状态。因 PPN 缺，PA 也无法成功转换。最终得

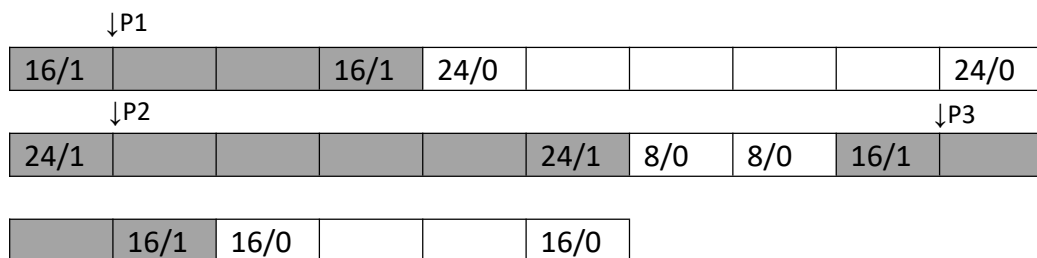
| Parameter         | Value     | Value     |
|-------------------|-----------|-----------|
| VA                | 0x014F2D3 | 0x07C01A5 |
| VPN               | 0x029E    | 0x0F80    |
| TLB index (TLBI)  | 0x2       | 0x0       |
| TLB Tag (TLBT)    | 0x0A7     | 0x3E0     |
| TLB Hit? (Y/N)    | Y         | Y         |
| Page Fault? (Y/N) | N         | N         |
| PPN               | 0x13      | -         |
| PA                | 0x09AD3   | -         |

（注：关系图如下）



## ★ Memory Allocation 内存分配

下图表示计算机内得堆 heap 区，其中每个块表示 1 个 word（4 个字节），灰色表示已经分配得块，白色表示空闲块，二者都有 Header 和 Footer，块中的 Header 和 Footer 已经在图中标记（各自占用 4 个字节）。分配器按照双字对齐。



假设采用“立即合并”的策略，分配空间时，会按照用户要求分割 free block，剩下的空间还可能再次利用。（P1 和 P3 的用户负载均为 8 字节，P2 为 16 字节。）  
程序按照行 1~行 6 运行以下内存分配操作的请求序列

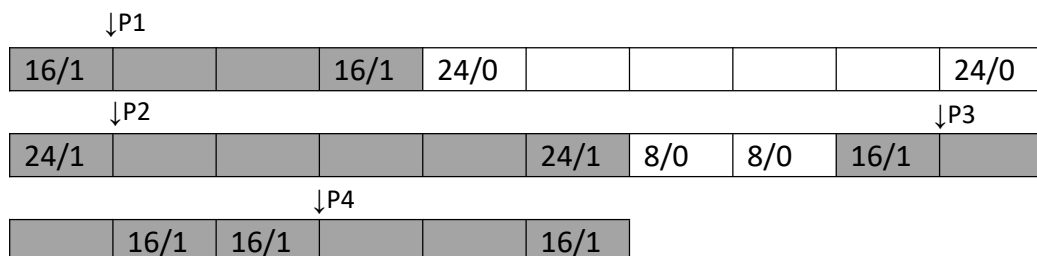
1. P4 = malloc(8);
2. free(P1);
3. P5 = malloc(6);
4. free(P2);
5. P6 = malloc(3);
6. P7 = malloc(2);

请回答以下问题，

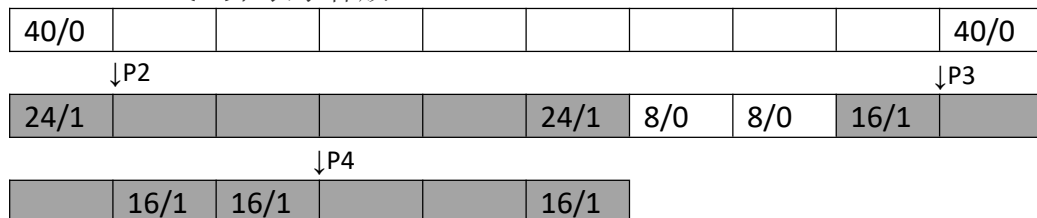
1，假设采用 best-fit 策略来寻址空间块。请画出执行完第 6 行代码后 heap 区内内存的状态图，标记出来每个块的 header 和 footer 的值。请计算此时内部碎片的总大小是多少字节。

答：best-fit 指寻找最合适大小的空闲块，优先分配，且原图表双字即 8 的倍数。

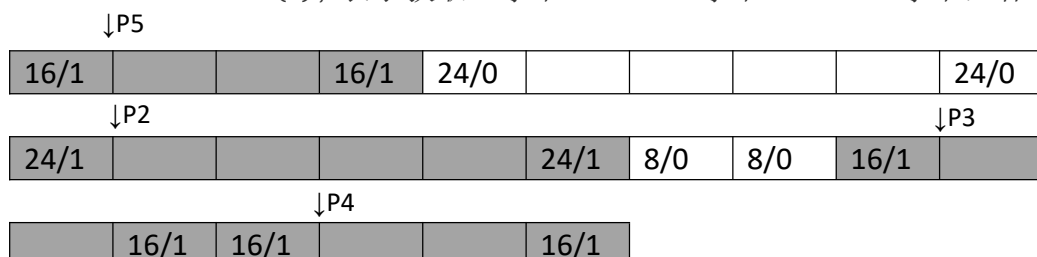
1. P4 = malloc(8); 表示获取 8 字节+题干中 header4 字节+footer4 字节



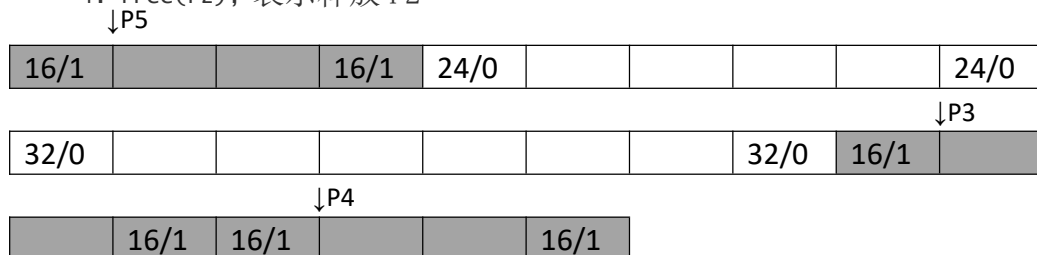
2. free(P1); 表示释放 P1



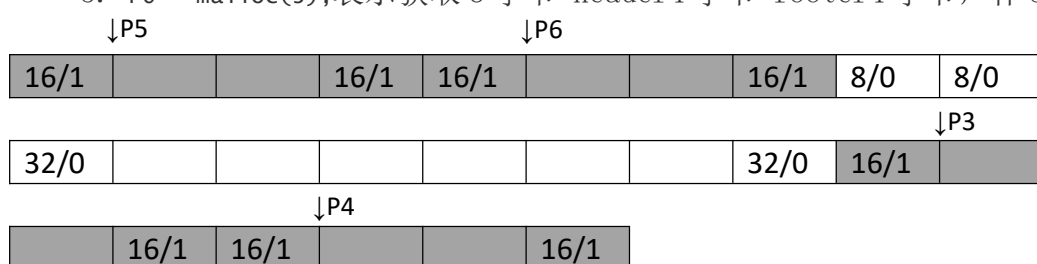
3. P5 = malloc(6); 表示获取 6 字节+header4 字节+footer4 字节，补 2 占 16



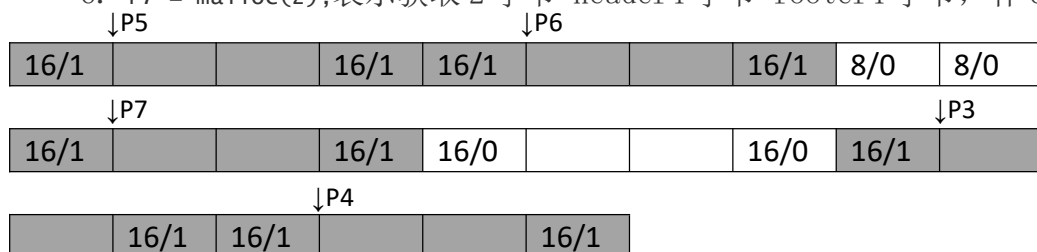
4. free(P2); 表示释放 P2



5. P6 = malloc(3); 表示获取 3 字节+header4 字节+footer4 字节, 补 5 占 16



6. P7 = malloc(2); 表示获取 2 字节+header4 字节+footer4 字节, 补 6 占 16



碎片计算, 碎片=header+footer+补, 即不可被利用空间总和

P3 = malloc(8); 表示获取 8 字节+题干中 header4 字节+footer4 字节 (注: 题干中标注负载是 8 字节)

P3 碎片=header+footer=8

P4 = malloc(8); 表示获取 8 字节+题干中 header4 字节+footer4 字节

P4 碎片=header+footer=8

P5 = malloc(6); 表示获取 6 字节+header4 字节+footer4 字节, 补 2 占 16

P5 碎片=header+footer+补=10

P6 = malloc(3); 表示获取 3 字节+header4 字节+footer4 字节, 补 5 占 16

P6 碎片=header+footer+补=13

P7 = malloc(2); 表示获取 2 字节+header4 字节+footer4 字节, 补 6 占 16

P7 碎片=header+footer+补=14

最后总计, 内部碎片大小为 8+8+10+13+14 = 53 字节

2. 假设采用 first-fit 策略来寻找空闲块。请画出执行完第 6 行代码后 heap 区内存的状态图, 标记出来每个块的 header 和 footer 的值。请计算此时内部碎片的总大小是多少字节。

Diagram illustrating the distribution of 16 processors across three rows of nodes:

- Row 1 (Top):** 10 nodes. Processors are distributed as follows: 16/1, (empty), (empty), 16/1, 16/1, (empty), (empty), 16/1, 8/0, 8/0. Arrows labeled P1 and P4 point to the first and fourth nodes respectively.
- Row 2 (Middle):** 10 nodes. Processors are distributed as follows: 24/1, (empty), (empty), (empty), (empty), 24/1, 8/0, 8/0, 16/1, (empty). Arrows labeled P2 and P3 point to the first and ninth nodes respectively.
- Row 3 (Bottom):** 6 nodes. Processors are distributed as follows: (empty), 16/1, 16/0, (empty), (empty), 16/0.

Diagram illustrating the state of the array after the third iteration:

|      |  |  |      |      |  |  |      |     |     |
|------|--|--|------|------|--|--|------|-----|-----|
| 16/0 |  |  | 16/0 | 16/1 |  |  | 16/1 | 8/0 | 8/0 |
|------|--|--|------|------|--|--|------|-----|-----|

Annotations:

- ↓P4 points to the 5th slot (16/1).
- ↓P2 points to the 1st slot (16/0).
- ↓P3 points to the 9th slot (16/1).

Below the array, a new array is shown:

|  |      |      |  |  |      |
|--|------|------|--|--|------|
|  | 16/1 | 16/0 |  |  | 16/0 |
|--|------|------|--|--|------|

Diagram illustrating memory layout and pointers:

- Row 1: 16/1, 16/1, 16/1, 8/0, 8/0. Pointers P5 and P4 point to the first and third blocks respectively.
- Row 2: 24/1, 24/1, 8/0, 8/0, 16/1. Pointers P2 and P3 point to the first and fifth blocks respectively.
- Row 3: 16/1, 16/0, 16/0.

Diagram illustrating memory layout and access patterns:

- Row 1: 16/1, (grey), 16/1, 16/1, (grey), (grey), 16/1, 8/0, 8/0. Accesses P5 and P4 are indicated above the first and fourth blocks.
- Row 2: 32/0, (white), (white), (white), (white), (white), (white), 32/0, 16/1, (grey). Access P3 is indicated above the ninth block.
- Row 3: (grey), 16/1, 16/0, (white), (white), 16/0.

Diagram illustrating the evolution of a 10-cell array over three steps, showing the movement of pointers P3, P4, and P5.

**Step 1 (Initial State):**

- Array: 

|      |  |  |      |      |  |  |      |     |     |
|------|--|--|------|------|--|--|------|-----|-----|
| 16/1 |  |  | 16/1 | 16/1 |  |  | 16/1 | 8/0 | 8/0 |
|------|--|--|------|------|--|--|------|-----|-----|
- Pointers: P5 points to the first cell (16/1), P4 points to the fourth cell (16/1).

**Step 2:**

- Array: 

|      |  |  |      |      |  |  |      |      |  |
|------|--|--|------|------|--|--|------|------|--|
| 16/1 |  |  | 16/1 | 16/0 |  |  | 16/0 | 16/1 |  |
|------|--|--|------|------|--|--|------|------|--|
- Pointers: P6 points to the first cell (16/1), P3 points to the ninth cell (16/1).

**Step 3:**

- Array: 

|  |      |      |  |  |      |
|--|------|------|--|--|------|
|  | 16/1 | 16/0 |  |  | 16/0 |
|--|------|------|--|--|------|
- Pointers: P6 points to the second cell (16/1), P3 points to the sixth cell (16/0).

The diagram illustrates three rows of memory blocks. Each block is a rectangle containing a text label. The first row contains eight blocks: '16/1', 'P5', '16/1', '16/1', 'P4', '16/1', '8/0', and '8/0'. The second row contains eight blocks: '16/1', 'P6', '16/1', '16/1', 'P7', '16/1', '16/1', and 'P3'. The third row contains three blocks: '16/1', '16/0', and '16/0'. The labels 'P3', 'P4', 'P5', 'P6', and 'P7' are positioned above their respective blocks, with a downward arrow pointing to each.

碎片计算，碎片=header+footer+补，即不可被利用空间总和

P3 = malloc(8); 表示获取 8 字节+题干中 header4 字节+footer4 字节（注：题干中标注负载是 8 字节）

P3 碎片=header+footer=8

P4 = malloc(8); 表示获取 8 字节+题干中 header4 字节+footer4 字节

P4 碎片=header+footer=8

P5 = malloc(6); 表示获取 6 字节+header4 字节+footer4 字节，补 2 占 16

P5 碎片=header+footer+补=10

P6 = malloc(3); 表示获取 3 字节+header4 字节+footer4 字节，补 5 占 16

P6 碎片=header+footer+补=13

P7 = malloc(2); 表示获取 2 字节+header4 字节+footer4 字节，补 6 占 16

P7 碎片=header+footer+补=14

↓P6

最后总计，内部碎片大小为 8+8+10+13+14 = 53 字节

### ★ 双单字对齐（单 4 双 8，头 4 脚 4）

| 对齐要求 | 已分配块  | 空闲块   | 最小块大小（字节）                        |
|------|-------|-------|----------------------------------|
| 单字   | 头部，脚部 | 头部和脚部 | $(4+x+4)/4$ 为整数，因此 x 最小 4，块最小 12 |
| 单字   | 头部，无脚 | 头部和脚部 | $(4+x)/4$ 为整数，因此 x 最小 4，块最小 8    |
| 双字   | 头部，脚部 | 头部和脚部 | $(4+x+4)/8$ 为整数，因此 x 最小 8，块最小 16 |
| 双字   | 头部，无脚 | 头部和脚部 | $(4+x)/8$ 为整数，因此 x 最小 4，块最小 8    |

### ★ 内存块计算

确定下面 malloc 请求序列的块大小和头部值。假设 1，双字对齐，使用隐式空闲链表。2，块大小四舍五入 8 字节倍数

| 请求         | 块大小（十进制） | 块头部（十六进制） |
|------------|----------|-----------|
| malloc(3)  |          |           |
| malloc(11) |          |           |
| malloc(20) |          |           |
| malloc(21) |          |           |

解：按照上一题内存分配里的知识

P6 = malloc(3); 表示获取 3 字节+header4 字节+footer4 字节，补 5 占 16

又因为，题干只有 malloc 请求序列的块大小和头部值，因此本题忽略尾部

即此题 malloc(3)，表示取 3 字节，头部 4，补 1 占 8（题干要求 8 得倍数）

块头部指块头部位置，按照上一题知识，头部是 8/1 开始，又因为 8 为十进制，

要在十进制，十六进制，二进制之间进行转换计算，对照十六转二表：

|    |      |      |      |      |      |      |      |      |
|----|------|------|------|------|------|------|------|------|
| 十六 | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
| 二  | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 |
| 十六 | 8    | 9    | A    | B    | C    | D    | E    | F    |
| 二  | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

8 为十进制，十六进制也为 8，因此 8/1 即 1000 与 0001 或运算，即 1001，1001 对应 9，因此头部 8/1 得十六进制为 0x09

同理 malloc(11)，表示取 11 字节，头部 4，补 1 占 16，得块大小为 16；  
16 是十进制，如何转十六进制，用除 16 得余数得方法。16/16=1，表示商为 1，余数为 0（此 0 为最低位，0 转十六进制转二进制即 0000）；  
把商 1 再次除 16，1/16=0，表示商为 0，余数为 1（商为 0，表示达到最高位，余数 1，转十六进制再转二进制即 0001）  
16 转十六转二为 00010000，那么头部 16/1 即 00010000 与 0001 或运算，即 00010001 得 0x11

同理 malloc(20) 表示 20 字节，头部 4，得 24  
24 是十进制，转十六，24/16=1，表示商为 1，余数为 8（最低位 1000）  
1/16=0，表示商为 0，余数为 1（最高位 0001）  
20 转十六转二为 00011000，那么头部 20/1 即 00011000 与 0001 或运算，即 00011001 得 0x19

同理 malloc(21) 表示 21 字节，头部 4，补 6，得 32  
32 是十进制，转十六，32/16=2，表示商为 2，余数为 0（最低位 0000）  
2/16=0，表示商为 0，余数为 2（最高位 0010）  
21 转十六转二为 00100000，那么头部 21/1 即 00100000+0001 即 00100001 得 0x21  
最终得

malloc(3)，块 8，头部 0x09

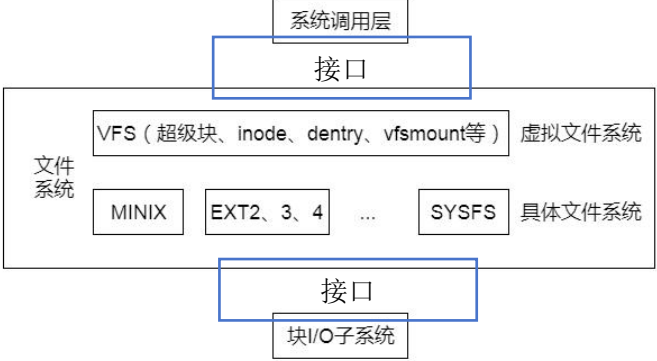
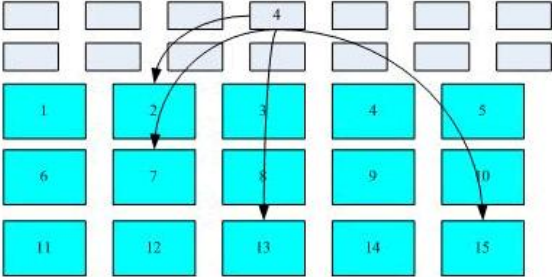
malloc(11)，块 16，头部 0x11

malloc(20)，块 24，头部 0x19

malloc(21)，块 32，头部 0x21

## 四，文件管理

电脑上总有些小电影告诉我们人生的意义，为了感恩这些老师，我们都有共同的希望（持久性）。小电影或许按照类型分类，或许老师姓名，年龄，职业等，这些只是我们看到的**形式（逻辑结构）**，是虚拟的，不是物理的，物理硬盘被划分多个块（每个块对应1个或数个扇区，扇区是硬盘上的物理基础单位），我们找老师们的时候手速都很快，为什么？因为**指针**！

|     |                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 持久性 | <p>★文件系统</p> <p>存储设备上组织文件的方法。</p> <p>操作系统中负责管理和存储文件信息的软件称为文件管理系统，简称文件系统。</p> <p>三部分组成，文件系统<b>接口</b>，对象操控与管理的<b>软件集合</b>，<b>对象和属性</b>。（注：上述出自《深入理解计算机系统》）</p> |  <p>Linux 虚拟文件系统四大对象，VFS 包含超级块，inode 索引节点，dentry 目录项，文件对象等（注：老师 ppt 中文件管理出自 linux 系统的文件管理，与左边教材不冲突。接口统一有助于面对复杂的硬件与系统调用）</p>                                                                                                                                                      |
|     | <p>★inode</p> <p>记录文件系统对象的元信息数据，简称索引节点，本质上是一个结构体。（注：就是文件属性相关信息）</p>                                                                                          | <pre>wrn3552@novadev:~/playground\$ stat file 文件: file 大小: 0          块: 0          IO 块: 4096   普通空文件 设备: fe21h/65057d   Inode: 32828      硬链接: 2 权限: (0644/-rw-r--r--)  Uid: ( 3041/ wrn3552)  Gid: ( 3041/ wrn3552) 最近访问: 2021-04-25 11:07:59.603745534 +0800 最近更改: 2021-04-25 11:07:59.603745534 +0800 最近改动: 2021-04-25 11:08:04.739848692 +0800 创建时间: -</pre> |
|     | <p>★dentry</p> <p>目录项，记录文件名字，目录结构，hardlink（hardlink 指向对应 inode）</p>                                                                                          | <pre>wrn3552@novadev:~/playground\$ tree . ├── dir │   └── file_in_dir ├── file └── hardlink</pre>                                                                                                                                                                                                                                                                 |
|     | <p>★指针 (indirect)</p> <p>指针定位文件位置，指针允许文件系统快速访问文件，指针指向对应数据块</p>                                                                                               |                                                                                                                                                                                                                                                                                |

## ★ VSFS (Very Simple File System)

搞懂这个知识点，先要了解操作系统如何与硬盘配合实现数据读写。

机械硬盘如下图，由很多个圆盘叠加组成，每个圆盘被划分很多个扇区

扇区：存储器存储单位，一般 1 扇区 4KB；

块：1 个块包含多个扇区，一般 1 块 16 个扇区；

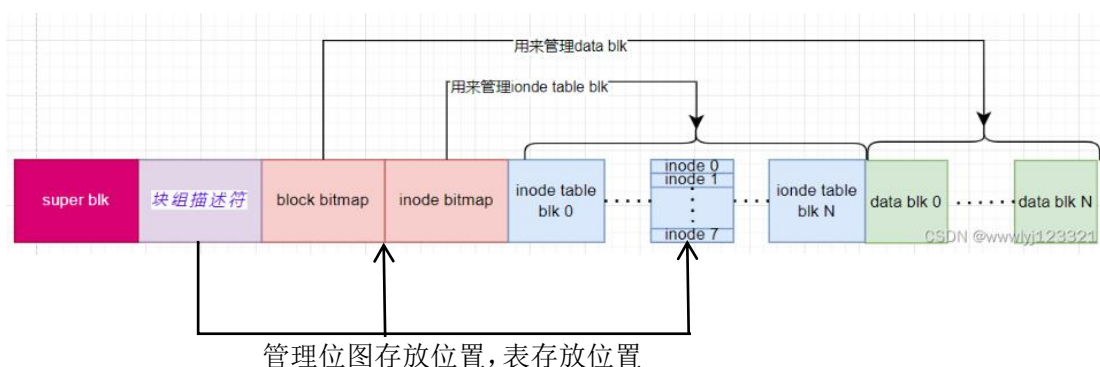


块作为基本单位，又按其作用，分为如下 6 类：

电脑上硬盘进行文件系统格式化后，会产生如下几个块



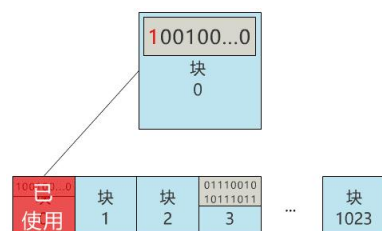
他们之间的关系如下图



1, 超级块 Superblock: 硬盘是否能够被读写取决于它；

2, 块描述符: 位图, 表的位置定位

3, Block bitmap 块位图: 管理块用一串 0 与 1 表示数据块是否被占用；

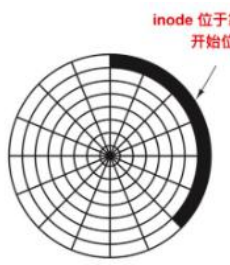


4, Inodebitmap inode 位图: 管理每条 inode 是否被占用；

5, Inode 表: 也称索引节点块 inode table blocks, 该表包含数条 inode 信息，

|             | Inode Number | File Type | Permission | Link count | UID | GID | size | ... | pointer |
|-------------|--------------|-----------|------------|------------|-----|-----|------|-----|---------|
|             | 1            | -         | 644        | 1          | 500 | 500 |      |     |         |
| inode Table | 2            | d         | 755        | 1          | 0   | 0   |      |     |         |
|             | ...          | ...       | ...        | ...        | ... | ... | ...  | ... | ...     |
|             | n            | -         | 644        | 2          | 501 | 501 |      |     |         |

每条 inode 又包含如下信息：



```

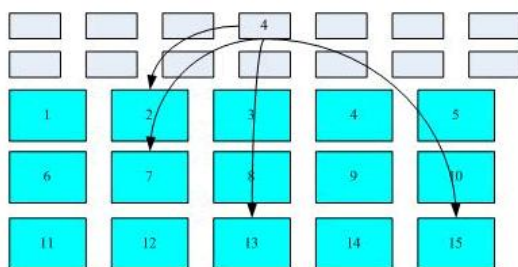
wrn3552@novadev:~/playground$ stat file
 文件: file
 大小: 0          块: 0          IO 块: 4096   普通空文件
设备: fe21h/65057d    Inode: 32828      硬链接: 2
权限: (0644/-rw-r--r--)  Uid: ( 3041/ wrn3552)   Gid: ( 3041/ wrn3552)
最近访问: 2021-04-25 11:07:59.603745534 +0800
最近更改: 2021-04-25 11:07:59.603745534 +0800
最近改动: 2021-04-25 11:08:04.739848692 +0800
创建时间: -

```

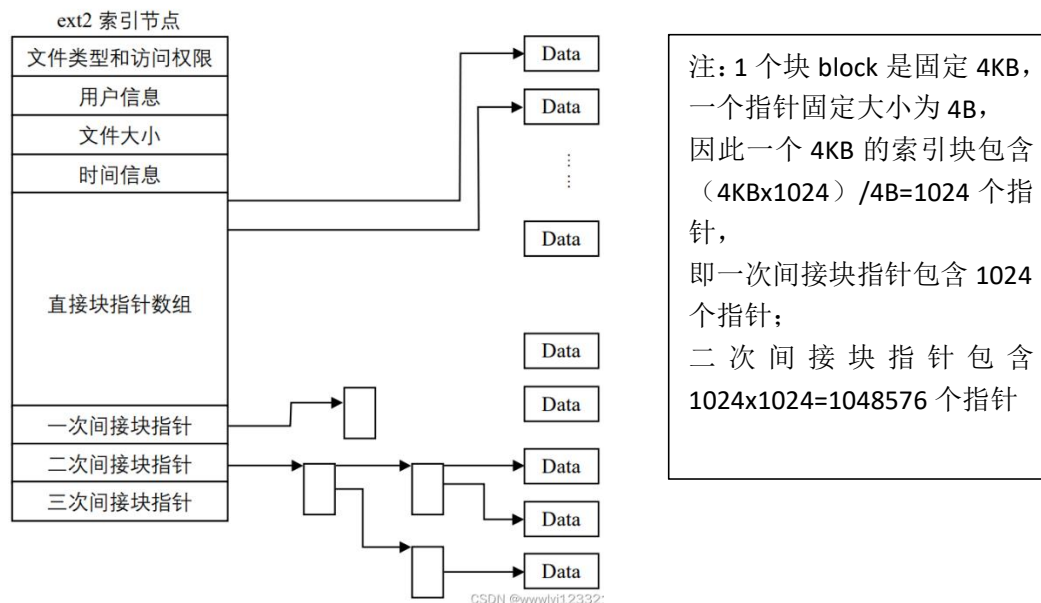
上图可以看出，每条 inode 包含了文件的属性信息，包括文件类型，大小，文件包含数据块数量，指针，权限，日期等（文件名不存在 inode 中）；

6，数据块 data blocks: 可以保存文件实际数据，也可以保存数据块块号（索引空间）就这两个（文件名不存在数据块中）

举例，当我们存放小电影时，小电影会占用多个数据块，还会占用 1 个 inode，（1 个文件配唯一 1 个 inode，inode 类似文件数据块索引指南）。假设这个小电影被分配了 4 个数据块，那么对应的 inode 包含了 4 个指针，指向这 4 个数据块。如下图



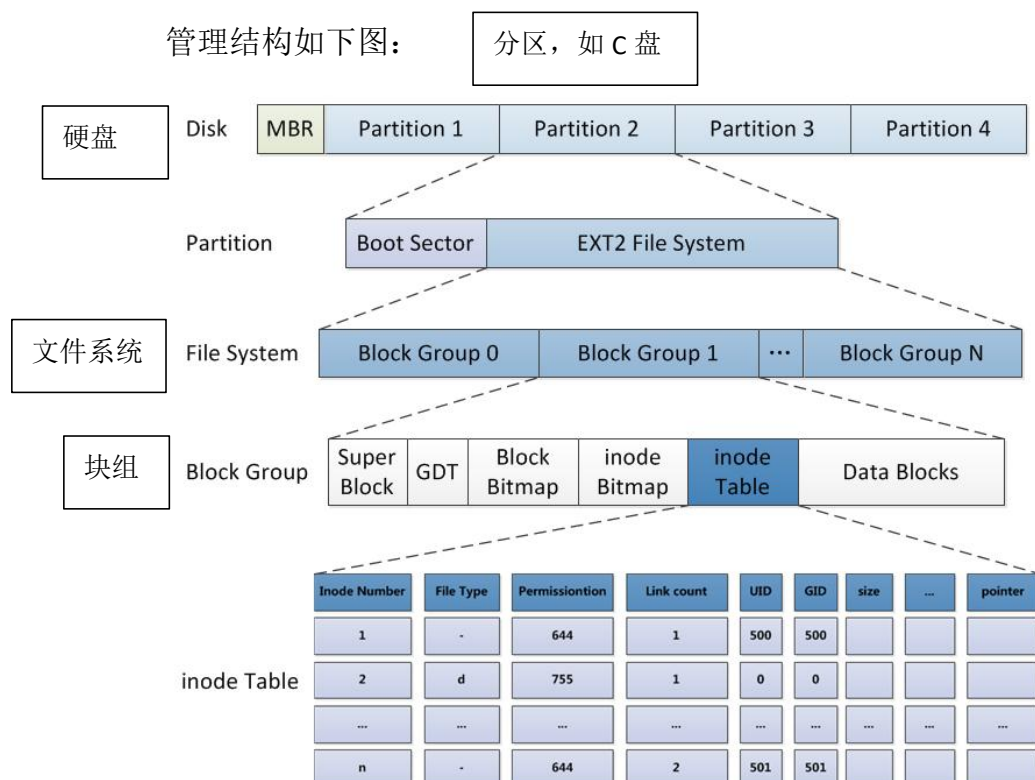
但是，问题来了，如果小电影高清无码蓝光贼大，比如 1 个 G，我们 1 个数据块一般 4KB，也就是需要 2.68 亿个数据块存放，也就是需要 2.68 亿个指针，1 个 inode 绝对没有这么多指针，怎么办？解决方案是间接索引，如下图，



我们现在总结下: 文件系统是一个分级管理的结构

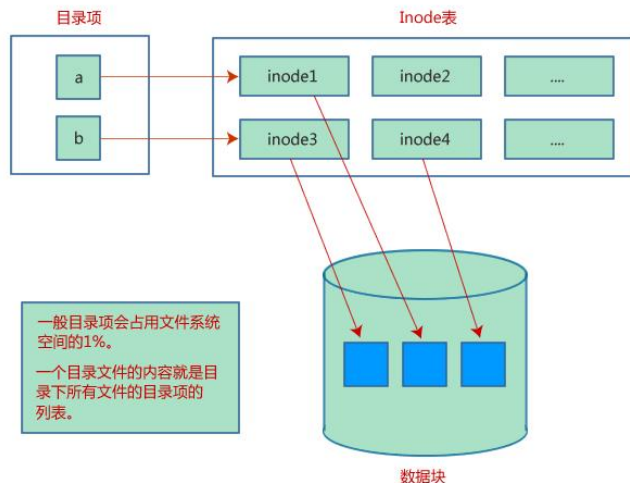
- 1, Linux 系统, 文件管理的 VFS 部分包含, 超级块, 目录项, inode, 文件对象;
- 2, 硬盘格式化, 生成超级块, 块描述, 块位图, inode 位图, inode 表, 数据块;
- 3, inode 表包含多条 inode 信息。数据块包含文件对象或块编号;
- 4, 一个文件对象, 对应 1 个 inode, inode 包含文件属性+直接指针+间接指针, inode 只知文件对应数据块号, 文件权限, 文件大小等, 唯独不知道文件名;
- 5, 直接指针指向数据块, 间接指针指向索引空间 (即只含块编号的数据块), 一次间接块指针包含 1024 个指针, 二次间接指针包含  $1024 \times 1024$  个指针。

管理结构如下图:



之前一直提到 **dentry** 目录项 保存在哪里？**dentry** 管理什么？

上文提到，当硬盘格式化后，并没有 **dentry** 目录项，它是保存在内存中，负责管理三类信息，文件目录结构，文件名，文件与 **inode** 映射关系，如下图



**例：**假设 32 位计算机中，每个 data block 的大小是 4KB，inode 结构中包含 12 个 direct pointers（直接块指针），2 个 indirect point（一级间接块指针），1 个 double indirect（二级间接块指针），请问该文件系统支持一个文件最大位多少？

假设 1 个文件是 100KB，请画出指针结构。

解：前文知识点，1 个 block 对应 1 个 inode 指针。

因此我们只要计算多少个指针，就可以知文件大小。

1 个直接指针 12

2 个一级间接指针  $2 \times 1024$

1 个二级间接指针  $1024 \times 1024$

总计指针  $= 12 + 2048 + 1024 \times 1024 = 1050636$

文件大小  $= 1050636 \times 4KB = 4202544KB = 4104MB$

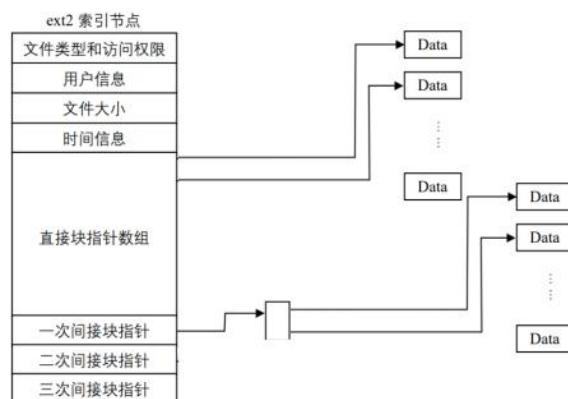
100KB 文件的数据块为  $100KB / 4KB = 25$  个

即得指针数 25 个

又因为直接指针 12 个，一级间接指针 1024 个

所以 100KB 文件会用到直接指针+一级间接指针

因此指针结构图如下



**例：**假设文件系统采用类似 Very Simple File System 的方式进行数据分布，包括 superblock, inode bitmap, databitmap, inodes（每个 inode 包含 12 个 direct, 2 个 indirect, 1 个 double indirect, 所有指针均为 4B）和 datablock, 每个文件或目录每次至少分配一个 block。Superblock, inode bitmap, databitmap 各占 1 个 block, 每个 block 的大小为 4KB。

假设以下两问中的操作是独立进行的，初试状态缓存中没有任何数据，访问一次即缓存；假设对文件的访问或修改需要在该文件、该文件所在文件夹记录访问/修改时间（再上层目录不受影响），所有修改会立刻落盘（对一个数据块的多次更新算消耗一次 I/O）。请计算下列操作需要进行多少次 I/O（包括读、写）？建议写出具体过程，每个部分的 I/O 次数，以便出错时能够得一部分分数。

**问：**文件系统中已有/home/ics/test 文件，大小为 2MB，读出该文件的全部内容。

**解：**补充知识，

其一，linux 下一切皆文件，目录也是文件，文件类型为目录。既然文件含有 1 个 inode，那么目录作为文件，自然也含有 1 个 inode；

其二，前文提到 dentry 目录项，电脑通电启动系统后，在内存生成的文件。比如文件系统中已有/home/ics/test 文件，那么就存在 3 个目录文件分别为：/, /home, /ics, 每个目录文件下的数据信息为该目录下的所有名称和对应 inode 块编号。

其三，查找文件时，从根目录起遍历名称，找到 home 对应 inode 编号，比如编号是 123，然后读取 123 号 inode，获取该 inode 对应的数据块，从这数据块中找到 ics 对应的 inode 号，比如 334，然后读取 334 号 inode，获取该 inode 对应数据块，从数据块中找到 test 对应的 inode 号，按此方法顺藤摸瓜。

请看题：

文件系统中已有/home/ics/test 文件，大小为 2MB，读出该文件的全部内容。

按照上述读取文件方法，步骤顺序如下

| 步骤 | 操作过程                           | 解释                        | I/O 次数 |
|----|--------------------------------|---------------------------|--------|
| 1  | read s                         | 读取超级块                     | 1      |
| 2  | read / inode                   | 读取根目录 inode，获取数据块位置       | 1      |
| 3  | read / data block              | 读取数据块，找到 home 对应 inode 编号 | 1      |
| 4  | read / home inode              | 读取 home inode，获取数据块位置     | 1      |
| 5  | read / home data block         | 读取数据块，找到 ics 对应 inode 编号  | 1      |
| 6  | read / home/ics inode          | 读取 ics inode，获取数据块位置      | 1      |
| 7  | read/ home/ics data block      | 读取数据块，找到 test 对应 inode 编号 | 1      |
| 8  | read/ home/ics/test inode      | 读取 test inode，获取数据块位置     | 2      |
| 9  | read/ home/ics/test data block | 读取数据块                     | 512    |
| 10 | write /home/ics/test inode     | 更新 inode 记录的最近访问日期        | 1      |
| 11 | write /home/ics inode          | 更新 inode 记录的最近访问日期        | 1      |
| 12 | write /home inode              | 更新 inode 记录的最近访问日期        | 1      |
|    |                                | 总计                        | 522    |

第 8 步，2MB 文件， $2 \times 1024 / 4 = 512$  个数据块，对应 512 个指针，直接指针数为 12，一级指针数为 1024，因此需要额外读 1 次一级指针+1 次 inode=2 次

**例：**一个文件系统的数据分布类似 VSFS，inode 包括 15 个指针（12 个 direct，2 个 indirect，1 个 double indirect）写入一个 10MB 的 /foo/bar 文件，分别采用 data journaling 和 metadata journaling，分别需要多少次 I/O？（除了文件 data block 外的都算元数据，每个 4KB block 的读或写都算一次 I/O，创建或写入同一文件只需要最终保存一次 metadata，向 log 中写一个 record 也算写入一次 I/O）

解：此题与上一题不同在于，此题是新建。

新建一个 /foo/bar 文件，10MB，调用 I/O 次数如下

| 步骤 | 操作过程                         | 解释                       | I/O 次数 |
|----|------------------------------|--------------------------|--------|
| 1  | read s                       | 读取超级块                    | 1      |
| 2  | read / inode                 | 读取根目录 inode，获取数据块位置      | 1      |
| 3  | read / data block            | 读取数据块，找到 foo 对应 inode 编号 | 1      |
| 4  | read / foo inode             | 读取 foo inode，获取数据块位置     | 1      |
| 5  | read / foo data block        | 读取数据块，获取剩余空间信息           | 1      |
| 6  | read / inode bitmap          | 读取 inode 位图，获取可用空 inode  | 1      |
| 7  | read/ block bitmap           | 读取块位图，获取可用空数据块           | 1      |
| 8  | write/ foo/bar 文件 inode      | 创建 inode 及指针             | 5      |
| 9  | write/ foo/bar 文件 data block | 写入 bar 文件                | 2560   |
| 10 | write/ foo inode             | 更新 foo inode             | 1      |
| 11 | write/ foo data block        | 更新 foo data block        | 1      |
| 12 | write/ inode bitmap          | 更新 inode 位图              | 1      |
| 13 | write/ block bitmap          | 更新块位图                    | 1      |
|    |                              | 总计                       | 2576   |

第 5 步，读取 foo 数据块，是因为 foo 作为目录项，数据块内容是包含的子文件名及对应 inode 编号，此处是为了添加 bar 文件名称及 inode 编号，做的准备操作，目的是保证有足够空间可以添加这个新信息。

第 9 步，写入文件 10MB，需要  $10 \times 1024 / 4 = 2560$  个数据块，即 2560 个指针  
 直接指针数为 12，2 个一级指针数为  $2 \times 1024$ ，1 个二级指针数  $1024 \times 1024$ （注：二级指针本质上是一个 1024 数据行的表，其每行派生指向对应 1024 个一级指针）  
 $2560 > 12 + 2 \times 1024$ ，  
 因此第 8 步需要创建 bar 文件的 inode 1 次 + 创建一级指针 2 次 + 创建二级指针 1 次 + 创建二级指针派生的一级指针 1 次，总计 5 次调用

第 11 步，此处更新 foo data block 是因为这个数据块包含了新建 bar inode 编号

**Datajournaling** 指数据日志，

日志开始 1 次，写入数据需要 2560 个数据块即 2560 次，日志提交 1 次  
 因此数据日志需额外调用 I/O 2562 次

**Metajournaling** 指元数据日志，

读取不需要修改日志，写入需要修改日志，因此

从上表第 8 步到 13 步，共计 9 次写入元数据

日志开始 1 次，日志提交 1 次

因此元数据日志需额外调用 I/O 11 次

**例：**假设文件系统采用类似 Very Simple File System 的方式进行数据分布，包括 superblock，inode bitmap，databitmap，inodes（每个 inode 包含 12 个 direct，2 个 indirect，1 个 double indirect，所有指针均为 4B）和 datablock，每个文件或目录每次至少分配一个 block。Superblock，inode bitmap，databitmap 各占 1 个 block，每个 block 的大小为 4KB。

假设以下两问中的操作是独立进行的，初试状态缓存中没有任何数据，访问一次即缓存；假设对文件的访问或修改需要记录在该文件、该文件所在文件夹记录访问/修改时间（再上层目录不受影响），所有修改会立刻落盘（对一个数据块的多次更新算消耗一次 I/O）。请计算下列操作需要进行多少次 I/O（包括读、写）？建议写出具体过程，每个部分的 I/O 次数，以便出错时能够得一部分分数。

问：文件系统中已有 /home/ics/test 文件，大小为 2MB，在文件尾部追加写入 30MB 数据。

解：与上一题不同，在于是追加，不是新建。

因为没有新增 inode，所以不需要读取 inode 位图，自然也不需要更新 inode 位图；因为没有新建 test inode，也不需要更新上级菜单 home data block。

这就是和上一题的主要区别。

| 步骤 | 操作过程                             | 解释                        | I/O 次数 |
|----|----------------------------------|---------------------------|--------|
| 1  | read s                           | 读取超级块                     | 1      |
| 2  | read / inode                     | 读取根目录 inode，获取数据块位置       | 1      |
| 3  | read / data block                | 读取数据块，找到 home 对应 inode 编号 | 1      |
| 4  | read / home inode                | 读取 home inode，获取数据块位置     | 1      |
| 5  | read / home data block           | 读取数据块，找到 ics 对应 inode 编号  | 1      |
| 6  | read / home/ics inode            | 读取 ics inode，获取数据块位置      | 1      |
| 7  | read/ home/ics data block        | 读取数据块，找到 test 对应 inode 编号 | 1      |
| 8  | read/ home/ics/test inode        | 读取 test inode，获取基本信息      | 1      |
| 9  | read/ block bitmap               | 读取块位图，获取可用空数据块            | 1      |
| 10 | write/ home/ics/test inode       | 更新 inode 及指针              | 10     |
| 11 | write/ home/ics/test data blocks | 写入追加数据                    | 7680   |
| 12 | write/ home/ics inode            | 更新 ics inode              | 1      |
| 13 | write/ home inode                | 更新 home inode             | 1      |
| 14 | write/ block bitmap              | 更新块位图                     | 1      |
|    |                                  | 总计                        | 7702   |

第 10 步， $30\text{MB} \times 1024 / 4 = 7680$ ，即需要 7680 个数据块

第 11 步，计算所需指针数， $7680 - 12 - 2 \times 1024 = 5620$

$5620 / 1024 = 5.4$ ，因此二级指针派生出 6 个一级指针

因此写入一级指针 2 次+二级指针 1 次+二级指针派生一级指针 6 次+更新 inode 1 次（注：这个更新包含直接指针及时间属性等信息），总计 10 次

# 五，I/O 输入输出管理

男人总是在追求大和持久这件事上孜孜不倦



图灵奖获得者  
David Patterson

教授很了不起，小宅男都应该感谢他，提出了 RAID 概念，即把廉价机械硬盘组成阵列，产生又大，又持久的效果，欧美风妥妥的。



但是，现代人口味变了，喜欢瘦，虽然不那么持久，但小奶狗一次输出够快，极致享受，但不耐久，容易玩坏。



选机械硬盘，还是 SSD 固态硬盘？作为成年人，面对两种完全不同硬件，大小不同，快慢有别，甚至不同国家的硬件，根本不用选，我都要，这时候绝不能说不行，双管齐下，还要互相配合，I/O 说我可以，你很行。

| 名词解释（★为考点）                                                           | 注释                        |
|----------------------------------------------------------------------|---------------------------|
| <p>★I/O 重定向</p> <p>允许用户将磁盘文件与标准输入输出联系起来（注：异常强大功能，调试，进程通信，自动化都靠它）</p> | <p>可以理解为不同程序，进程间的输出共享</p> |

## ★ 磁盘调度算法

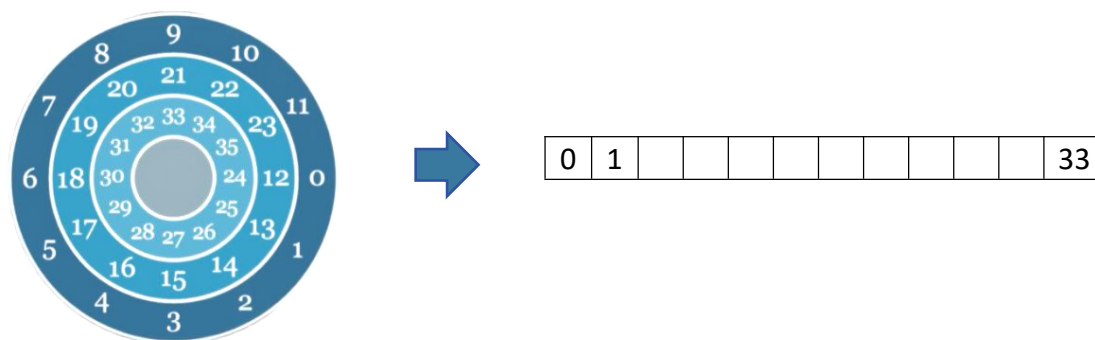
计算机每个扇区为 512B。以下 I/O 请求几乎同时、但按照顺序到达 I/O 调度层，其中的数字为扇区编号。当前磁头正处于 100 号扇区处，并向扇区编号较大的方向移动。

I/O 请求序列为{120, 90, 45, 170, 50, 75, 125, 30, 20}

假设计算机只有一块磁盘，没有进行 RAID 配置，所使用的磁盘调度策略分别为 First Come First Serve (FCFS)、Shortest Seek Time First (SSTF)、F-SCAN(磁头任意移动方向都可以服务 I/O 请求)、CSCAN 调度策略(只有磁头从小编号扇区向大编号扇区移动的过程中可以服务 I/O 请求)。

请分别给出四种策略的实际 I/O 请求服务序列，以及完成这些 I/O 请求的总寻道时间（假设每移动一个扇区，寻道时间加 1）。对于 SSTF，如果遇到寻道时间相同的请求，尽可能不改变磁头移动方向。

解：机械硬盘结构上划分了很多扇区，我们可以把它铺平排列，如下图



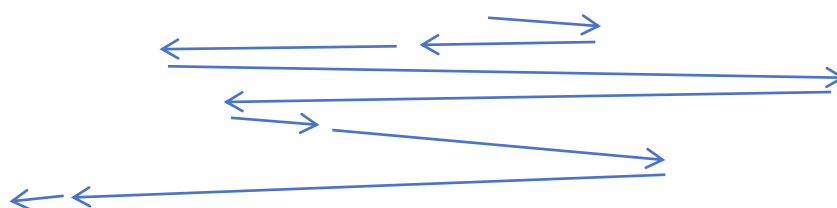
I/O 请求时，磁头就会在这行扇区上来回扫。

FCFS（先来先服务算法）

磁头处在 100 号扇区，对如下序列按序，依次扫

请求序列为{120, 90, 45, 170, 50, 75, 125, 30, 20}

|   |    |    |    |    |    |    |     |     |     |     |     |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|
| 0 | 20 | 30 | 45 | 50 | 75 | 90 | 100 | 120 | 125 | 170 | 200 |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|



寻道时间即扫描这些扇区的总时间，100 号扇区为起点，到 120 号扇区就需要 20 秒，依次类推，把所有差额累加即为总时间

|    |         |        |       |        |        |       |        |        |       |
|----|---------|--------|-------|--------|--------|-------|--------|--------|-------|
| 移动 | 100-120 | 120-90 | 90-45 | 45-170 | 170-50 | 50-75 | 75-125 | 125-30 | 30-20 |
| 差额 | 20      | 30     | 45    | 125    | 120    | 25    | 50     | 95     | 10    |

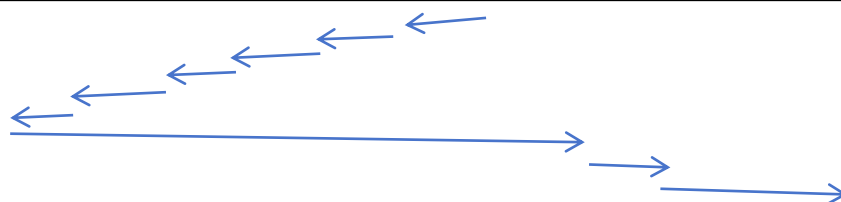
总时间 = 20+30+45+125+120+25+50+95+20=520

### SSTF（最短寻道时间优先算法）

磁头在 100 号扇区，每次都找离自己最近的扇区作为优先，扫描如下

请求序列为{120, 90, 45, 170, 50, 75, 125, 30, 20}

|   |    |    |    |    |    |    |     |     |     |     |     |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|
| 0 | 20 | 30 | 45 | 50 | 75 | 90 | 100 | 120 | 125 | 170 | 200 |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|



寻道时间即扫描这些扇区的总时间，100 号扇区为起点，到 90 号扇区就需要 10 秒，依次类推，把所有差额累加即为总时间

|    |        |       |       |       |       |       |        |         |         |
|----|--------|-------|-------|-------|-------|-------|--------|---------|---------|
| 移动 | 100-90 | 90-75 | 75-50 | 50-45 | 45-30 | 30-20 | 20-120 | 120-125 | 125-170 |
| 差额 | 10     | 15    | 25    | 5     | 15    | 10    | 100    | 5       | 45      |

总时间 = 10+15+25+5+15+10+100+5+45=230

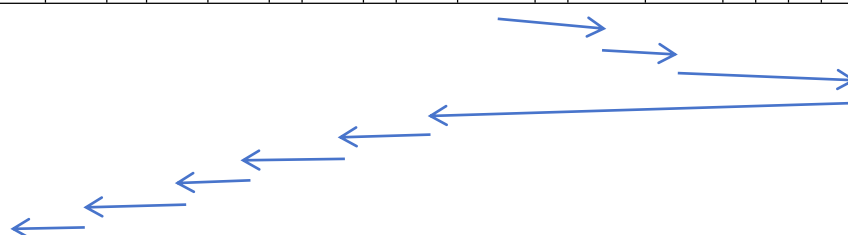
### F-SCAN（电梯调度法）

SSTF 算法可能导致磁头在一定时间内在某个小区域内徘徊，影响效率。因此在 SSTF 基础上规定，移动到最外侧，才能向内移动，移动到最内测，才能向外移动。

磁头在 100 号扇区，每次都找离自己最近的扇区作为优先，扫描如下

请求序列为{120, 90, 45, 170, 50, 75, 125, 30, 20}

|   |    |    |    |    |    |    |     |     |     |     |     |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|
| 0 | 20 | 30 | 45 | 50 | 75 | 90 | 100 | 120 | 125 | 170 | 200 |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|



寻道时间即扫描这些扇区的总时间，100 号扇区为起点，到 120 号扇区就需要 20 秒，依次类推，把所有差额累加即为总时间

|    |         |         |         |        |       |       |       |       |       |
|----|---------|---------|---------|--------|-------|-------|-------|-------|-------|
| 移动 | 100-120 | 120-125 | 125-170 | 170-90 | 90-75 | 75-50 | 50-45 | 45-30 | 30-20 |
| 差额 | 20      | 5       | 45      | 80     | 15    | 25    | 5     | 15    | 10    |

总时间 = 20+5+45+80+15+25+5+15+10=220

### C-SCAN（循环扫描算法）

磁头仅向外移动，到了最外请求扇区，直接返回最内请求扇区，再向外移动

请求序列为{120, 90, 45, 170, 50, 75, 125, 30, 20}

|   |    |    |    |    |    |    |     |     |     |     |     |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|
| 0 | 20 | 30 | 45 | 50 | 75 | 90 | 100 | 120 | 125 | 170 | 200 |
|---|----|----|----|----|----|----|-----|-----|-----|-----|-----|



寻道时间即扫描这些扇区的总时间，100 号扇区为起点，到 120 号扇区就需要 20 秒，依次类推，把所有差额累加即为总时间

|    |         |         |         |        |       |       |       |       |       |
|----|---------|---------|---------|--------|-------|-------|-------|-------|-------|
| 移动 | 100-120 | 120-125 | 125-170 | 170-20 | 20-30 | 30-45 | 45-50 | 50-75 | 75-90 |
| 差额 | 20      | 5       | 45      | 150    | 10    | 15    | 5     | 25    | 15    |

总时间 = 20+5+45+150+10+15+5+25+15=220=290

### ★ 磁盘调度算法优缺点

#### 1, SSTF 最短寻道时间

最近轨道上选择请求

缺点：饥饿（指请求调度，但长时间没得到响应）；陷入局部最优，达不到全局最优；

#### 2, F-Scan 电梯调度

由起始到外，再由外到内，一个方向扫描完才处理新来的请求

缺点：效率不高

优点：避免饥饿

#### 3, C-Scan 循环扫描

基于 F-Scan 的优化版，只从起始位到外，再从最内到最外。

优点：所有磁盘更平（指提高效率，磁盘利用率更平均）

### ★ FTL 算法 (SSD 固态硬盘)

固态硬盘 SSD 的 FTL 算法有个特点，只擦除，不覆盖，即当我们要修改数据并保存，只能在某个位置存入修改后的新数据，然后把老数据擦除。

SSD 内部 FTL 算法采用 hybrid mapping 的方式进行虚拟地址到物理地址的映射，block table 中记录逻辑 block id 与物理地址的映射关系，log table 记录逻辑 page id 与物理地址的映射关系（page id = 4\*block id）；

同时最多只能有 1 个 log block 存在，每个原始 block 对应一个 log block（即一个 log block 中的数据不能来自于不同的原始 block），选择 log block 进行合并操作时优先选择代价小的进行，选择空白 block 写入时采用 first-fit 原则；

初始状态如下图所示，到达的请求序列为 a, b, c, d, e, e, f, g, e, c，请画出序列结束后的状态，计算发生了几次 merge，分别是什么类型？

解：这类题是 FTL 映射简化版本，但依然要遵守如下规则：

1，同块合并：Log Table 只能接受同一个标号的 Block 数据，比如下表 Block0，你可以输入 abc，但一旦输入之外的数据，比如 e，就要进行 merge 操作；

|              |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|----------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | Switch merge               |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8（指原 250 号现指向 8），251-4 |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                          | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                          | b | c | d |  | e      | f | g | h |  | a'     | b' | c' | d  |  |        |    |    |    |
| Block        | Block0                     |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

2，满额合并：Block 是有数量限制的，比如下表的限制是 4，你输入新的 abcda，那在输入最后一个 a 之前要先进行 merge 操作；

|              |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|----------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | Switch merge               |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8（指原 250 号现指向 8），251-4 |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                          | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                          | b | c | d |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                     |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

Merge 操作分三类：

1，switch merge，如原数据 abcd，你按顺序输入 abcde，按照满额合并规则，输 e 时，会先进行 switch merge 操作，把原数据 abcd 擦除，保留新 abcd，如下表

|              |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|----------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | Switch merge               |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8（指原 250 号现指向 8），251-4 |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                          | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                          | b | c | d |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                     |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

2，Partial merge，如原数据 abcd，你输入 abef，按照同块合并规则，在输 e 的时候，要先进行 partial merge 操作，把原数据 cd 拷贝到新的 ab 后面，然后把原数据整个 Block 删除

|              |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|----------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | Partial merge              |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8（指原 250 号现指向 8），251-4 |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                            |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                          | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                          | b | c | d |  | e      | f | g | h |  | a'     | b' | c  | d  |  |        |    |    |    |
| Block        | Block0                     |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

3，Full merge，如原数据 abcd，你输入 bcef，按照同块合并规则，在输 e 的时候，要先进行 full merge 操作，把原数据 a 拷贝到新位置，新输入 bc 拷贝到新位置 a 后面，把原数据 d 拷贝到新位置 c 后面，然后删除两个 Block，如下表

|              |                              |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|------------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | Full merge                   |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   |                              |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-12（指原 250 号现指向 12），251-4 |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                              |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                            | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                            | b | c | d |  | e      | f | g | h |  | b'     | c' |    |    |  | a      | b' | c' | d  |
| Block        | Block0                       |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

基于上述规则与 merge 分类，我们开始解此题。

|              |                                                                     |   |   |   |  |        |   |   |   |  |        |   |    |    |  |        |    |    |    |
|--------------|---------------------------------------------------------------------|---|---|---|--|--------|---|---|---|--|--------|---|----|----|--|--------|----|----|----|
| 操作           | 最初                                                                  |   |   |   |  |        |   |   |   |  |        |   |    |    |  |        |    |    |    |
| Log Table:   |                                                                     |   |   |   |  |        |   |   |   |  |        |   |    |    |  |        |    |    |    |
| Block Table: | 250-0, 251-4（注：250 指 abcd 对应硬盘所在位置的数据块编号，251 指 efgh 对应硬盘所在位置的数据块编号） |   |   |   |  |        |   |   |   |  |        |   |    |    |  |        |    |    |    |
| -----        |                                                                     |   |   |   |  |        |   |   |   |  |        |   |    |    |  |        |    |    |    |
| Page         | 0                                                                   | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9 | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                                                                   | b | c | d |  | e      | f | g | h |  |        |   |    |    |  |        |    |    |    |
| Block        | Block0                                                              |   |   |   |  | Block1 |   |   |   |  | Block2 |   |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c（蓝字为当前输入项）

|              |                                                                                                                                                                                               |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 依次输入新 abcd，存在 Block2 上<br>（注：a'表示新输入的 a，存放在空的 block）                                                                                                                                          |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1000-8, 1001-9, 1002-10, 1003-11<br>（注：LogTable 表示当前最新数据存放在哪个位置，page id = 4*block id 为题干规则，即 250x4=1000，8 为块 Block2 开头的 Page 位置，1000-8 表示当前操作为 abcd 输入了新数据，新 a 在 8，新 b 在 9，新 c 在 10，新 d 在 11） |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-4（注：题是更新数据，更新后 250 对应 abcd 指向 block2）                                                                                                                                              |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                                                                                                                                                                                               |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                                                                                                                                                                                             | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | a                                                                                                                                                                                             | b | c | d |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                                                                                                                                                                                        |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c（蓝字为当前输入项）

|              |                                                                                                    |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|----------------------------------------------------------------------------------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 擦除 Block0，完成第一次 merge，类型为 switch merge，输入 e<br>（注：e'表示新输入的 e，block0 清空后才可存放新数据 e'）<br>（注：按照同块合并规则） |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1004-0（注：原 251 的 e 此时新数据指向 0）                                                                      |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-4                                                                                       |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                                                                                                    |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                                                                                                  | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | e'                                                                                                 |   |   |   |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                                                                                             |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c (蓝字为当前输入项)

|              |                                            |     |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|--------------------------------------------|-----|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 输入 e<br>（注：e''表示新输入的 e）                    |     |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1004-1（注：1 代表当前 e 应指向 e''位置 Page1，此才为最新数据） |     |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-4                               |     |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                                            |     |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                                          | 1   | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | e'                                         | e'' |   |   |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                                     |     |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c (蓝字为当前输入项)

|              |                           |     |    |    |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|---------------------------|-----|----|----|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 依次输入 fg<br>（注：f'表示新输入的 f） |     |    |    |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1004-1, 1005-2, 1006-3    |     |    |    |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-4              |     |    |    |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                           |     |    |    |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                         | 1   | 2  | 3  |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | e'                        | e'' | f' | g' |  | e      | f | g | h |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                    |     |    |    |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c (蓝字为当前输入项)

|              |                                                                                                                          |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|--------------------------------------------------------------------------------------------------------------------------|---|---|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 完成第二次 merge，类型是 Full merge，即把 e'，f'，g'拷贝到 Block3，擦除 Block0，把 h 拷贝到 Block3，擦除 Block1，输入 e<br>（注：e'''表示新输入的 e）（注：按照同块合并规则） |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1004-0                                                                                                                   |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-12（注：更新后 251 指向 12）                                                                                           |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                                                                                                                          |   |   |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                                                                                                                        | 1 | 2 | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | e'''                                                                                                                     |   |   |   |  |        |   |   |   |  | a'     | b' | c' | d' |  | e''    | f' | g' | h  |
| Block        | Block0                                                                                                                   |   |   |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

请求序列为 a, b, c, d, e, e, f, g, e, c (蓝字为当前输入项)

|              |                                                                                                            |    |    |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
|--------------|------------------------------------------------------------------------------------------------------------|----|----|---|--|--------|---|---|---|--|--------|----|----|----|--|--------|----|----|----|
| 操作           | 完成第三次 merge, 类型是 Partial merge, 即把 f', g', h 拷贝到 Block0, 擦除 Block3, 输入 c<br>(注: c''表示新输入的 c) (注: 按照满额合并规则) |    |    |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Log Table:   | 1002-4<br>(注: page id = 4*block id 为题干规则, 所以 1000 是 a, 1001 是 b, 1002 是 c, 当前操作是输入新 c, 新 c 的位置在 Page4)     |    |    |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Block Table: | 250-8, 251-0 (注: 原 251 对应的 efgh 现在指向 block0 开头 0)                                                          |    |    |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| -----        |                                                                                                            |    |    |   |  |        |   |   |   |  |        |    |    |    |  |        |    |    |    |
| Page         | 0                                                                                                          | 1  | 2  | 3 |  | 4      | 5 | 6 | 7 |  | 8      | 9  | 10 | 11 |  | 12     | 13 | 14 | 15 |
|              | e'''                                                                                                       | f' | g' | h |  | c''    |   |   |   |  | a'     | b' | c' | d' |  |        |    |    |    |
| Block        | Block0                                                                                                     |    |    |   |  | Block1 |   |   |   |  | Block2 |    |    |    |  | Block3 |    |    |    |

## ★ 磁盘位移

下面的代码中访问的文件 numbers.txt 的内容是 ASCII 码 0123456789。你可以假设

read()函数是原子性的。

dup2 函数原型为:

```
/* dup2(oldfd, newfd) overwrites the entry in the per-process file table for newfd
with the entry
for oldfd. */
int dup2(int oldfd, int newfd);
```

read\_and\_print\_one.h 代码为:

```
#ifndef READ_AND_PRINT_ONE
#define READ_AND_PRINT_ONE
#include <stdio.h>
#include <unistd.h>
static inline void read_and_print_one(int fd) {      //定义 read_and_print_one 函数
    char c;                                         //按照单个字符读取
    read(fd, &c, 1);                               //每次读 1 个字符
    printf( "%c" , c);
    fflush(stdout);
}                                                    //此为自定义函数，每次打印 1 个字符，按序读取
```

请写出下列代码的屏幕输出，建议写一些中间过程。

```
#include "read_and_print_one.h"
#include <stdlib.h>
#include <fcntl.h>
int main() {
    int file1 = open( "numbers.txt" , O_RDONLY); //变量 file1，打开 txt，获取描述
    int file2;                                   //只是设置了变量 file2
    int file3 = open( "numbers.txt" , O_RDONLY); //变量 file3，打开 txt，获取描述
    file2 = dup2(file3, file2);                  //把 file3 内容复制给 file2，但共享偏移量
    read_and_print_one(file1);                   //因自定义函数，只读 1 字符并打印 0
    read_and_print_one(file2);                   //因自定义函数，只读 1 字符并打印 0
    read_and_print_one(file3);                   //按序读取，接 file2，从 0 之后，打印 1
    //file3 与 file2 共享偏移量，file2 读取后自动偏移 1 字符，file3 按序后读
    read_and_print_one(file2);                   //按序读取，接 file3，从 1 之后，打印 2
    read_and_print_one(file1);                   //按序读取，从 0 之后，打印 1
    read_and_print_one(file3);                   //按序读取，接 file2，从 2 之后，打印 3
    return 0;                                    //退出
}
```

屏幕输出为: 001213